

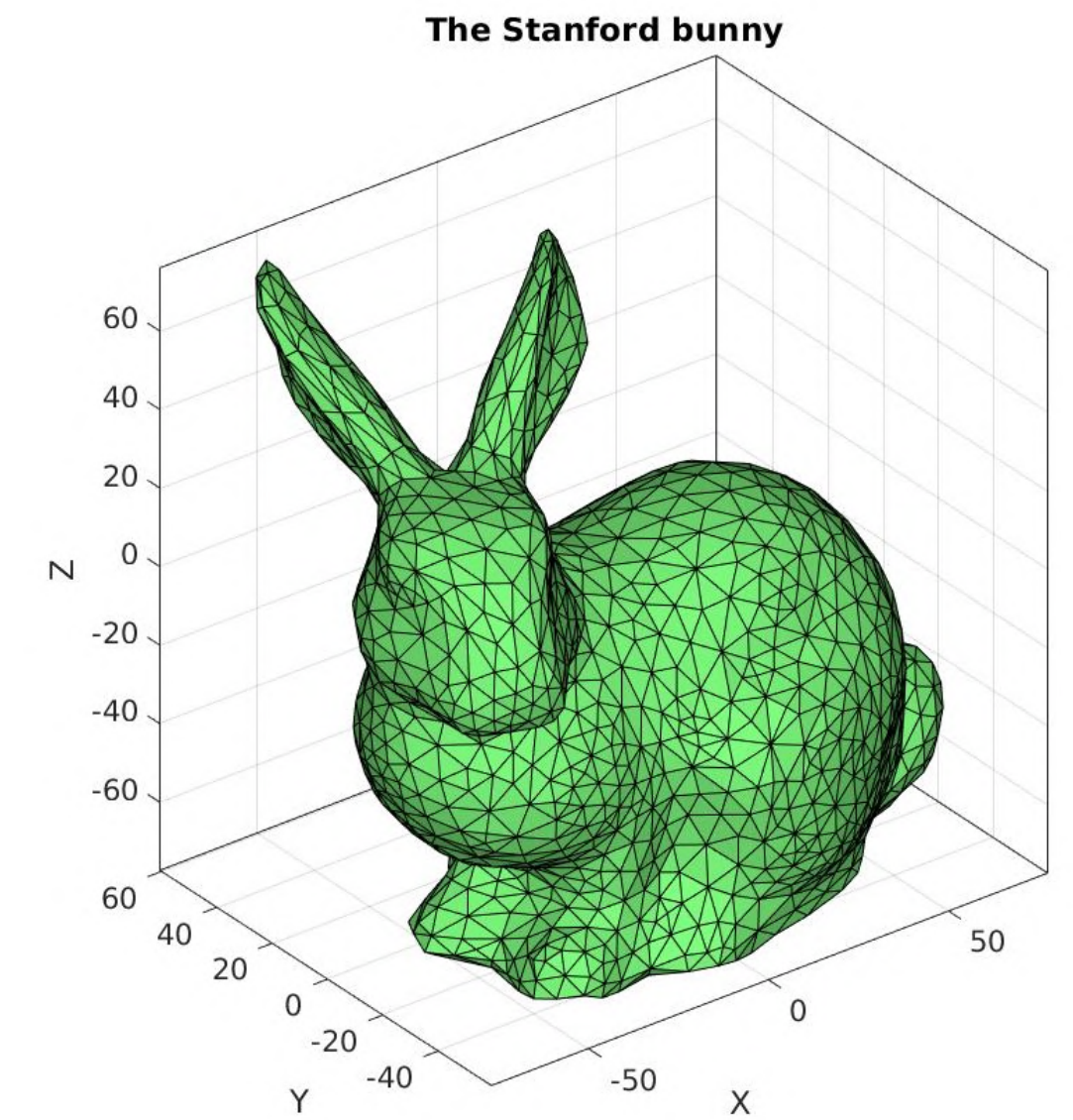
VIII: Implicit Representations for 3D

3D CV

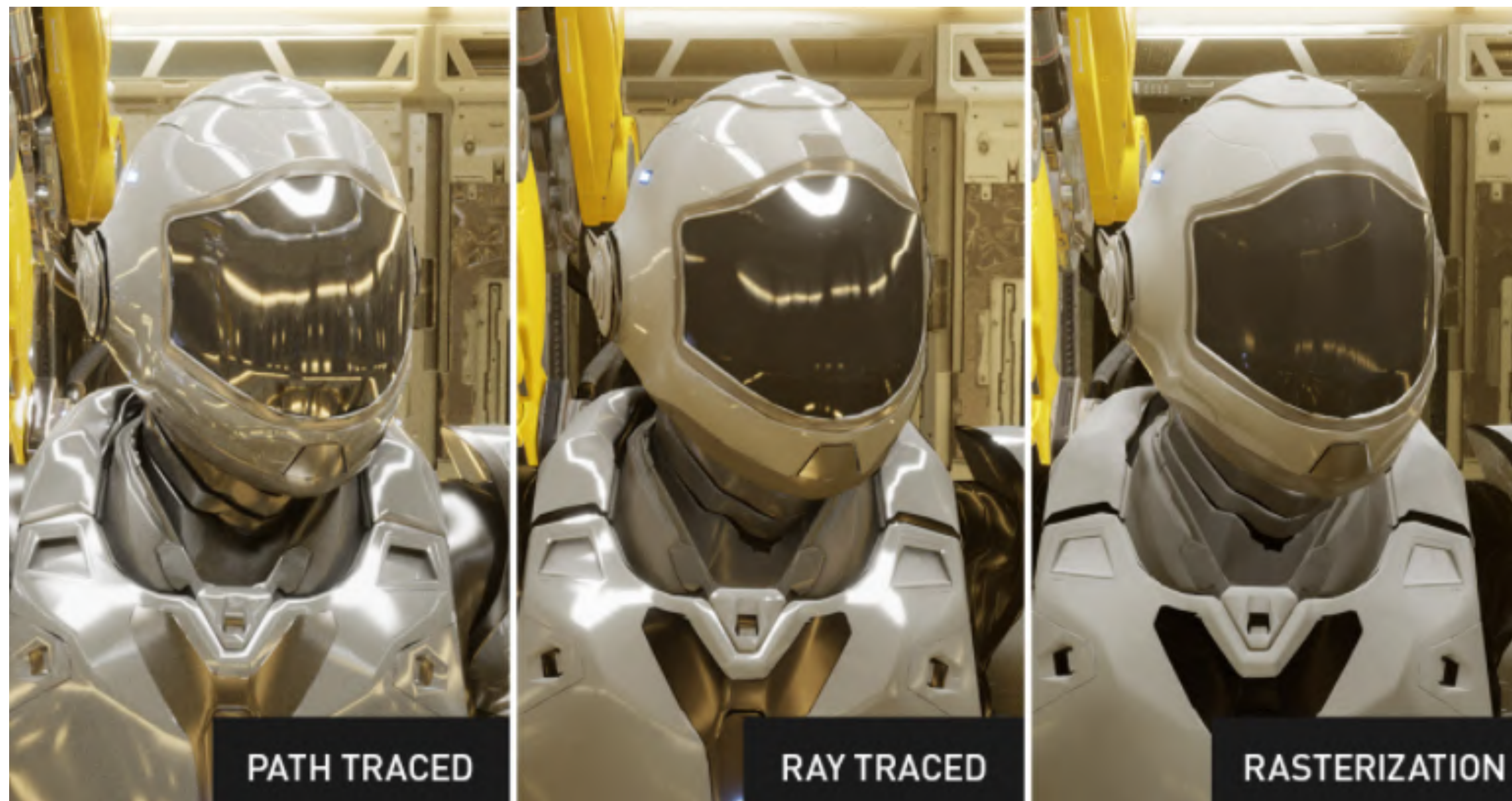
Kirill Struminsky, Mishan Aliev

In the Previous Episodes

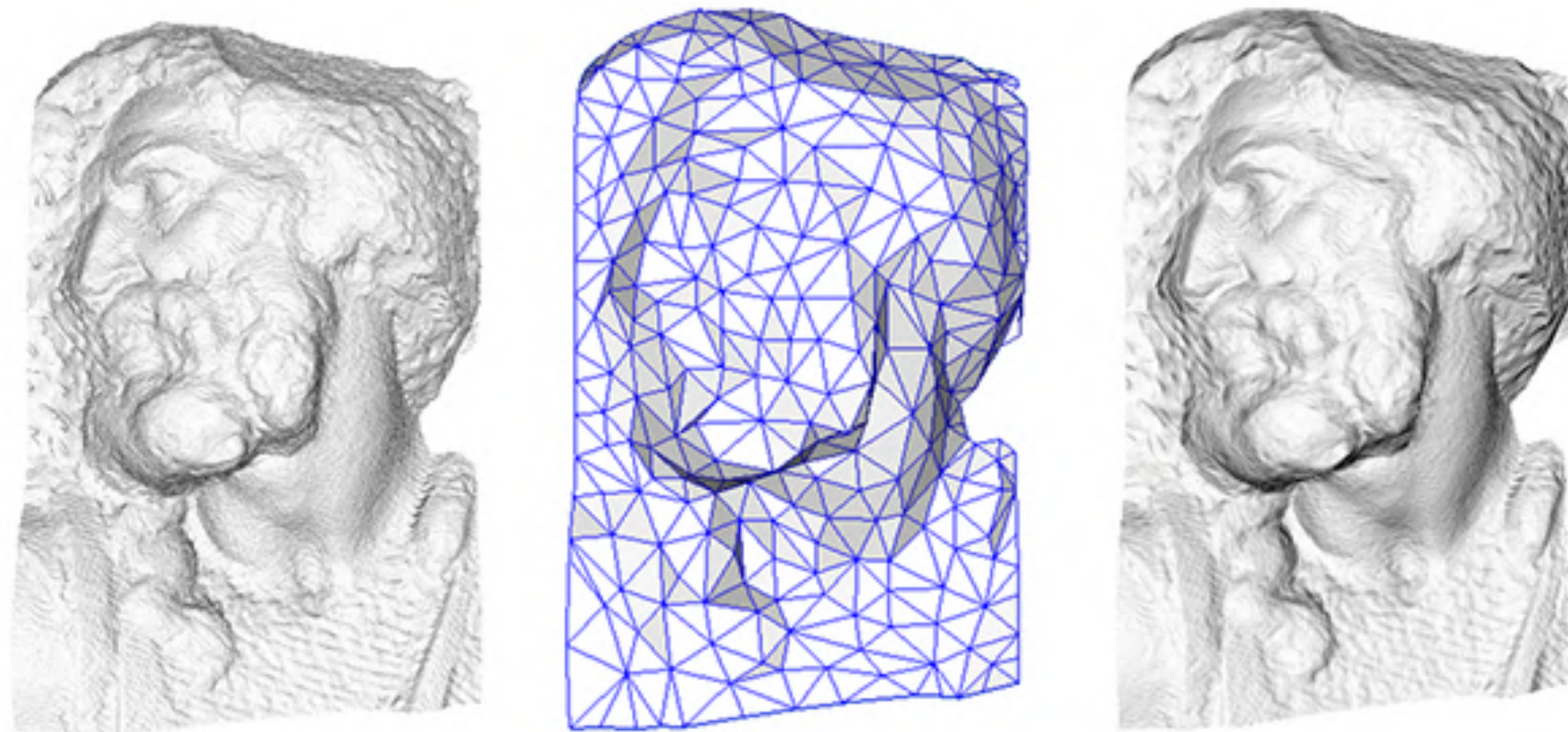
- Triangular meshes for surface representations
- Rendering algorithms
 - Path tracing
 - Rasterization
- Mesh models for human bodies
- Today: volumetric representations for 3D and beyond



Path Tracing versus Rasterisation



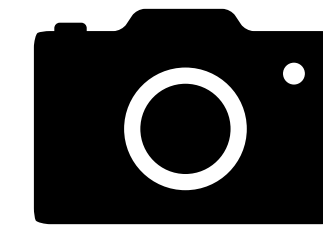
Radiance Functions



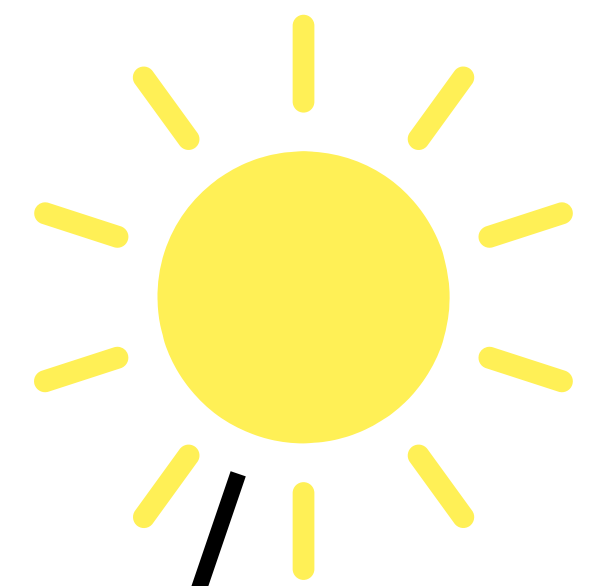
original mesh
4M triangles

simplified mesh
500 triangles

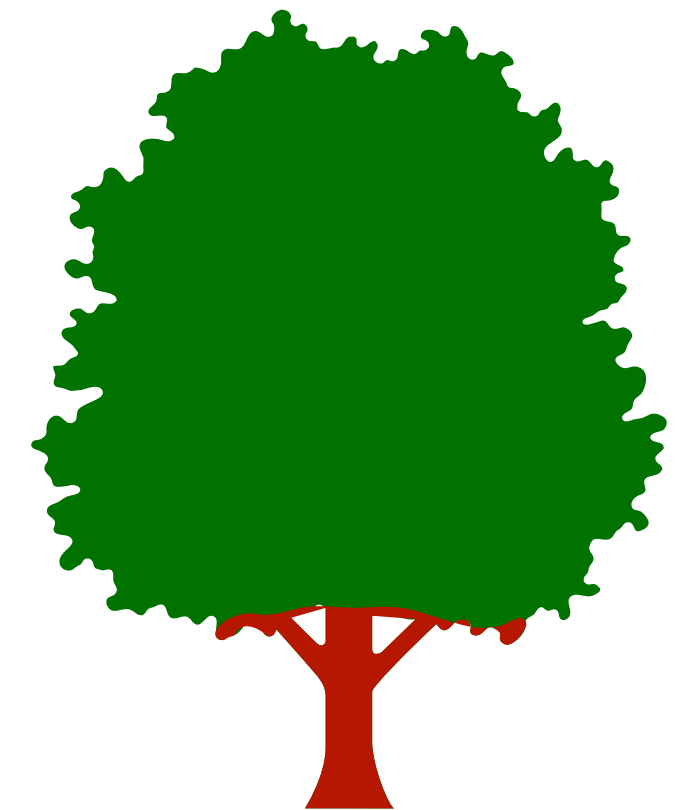
simplified mesh
and normal mapping
500 triangles



$$L_i(x_c, \omega_c) = L_o(x_t, -\omega_c)$$



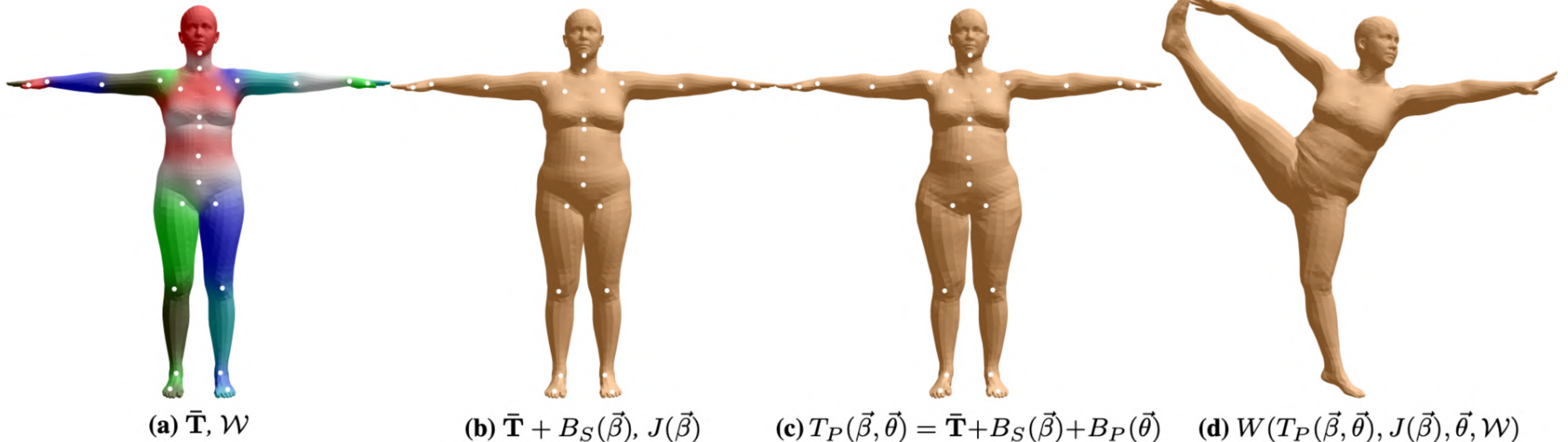
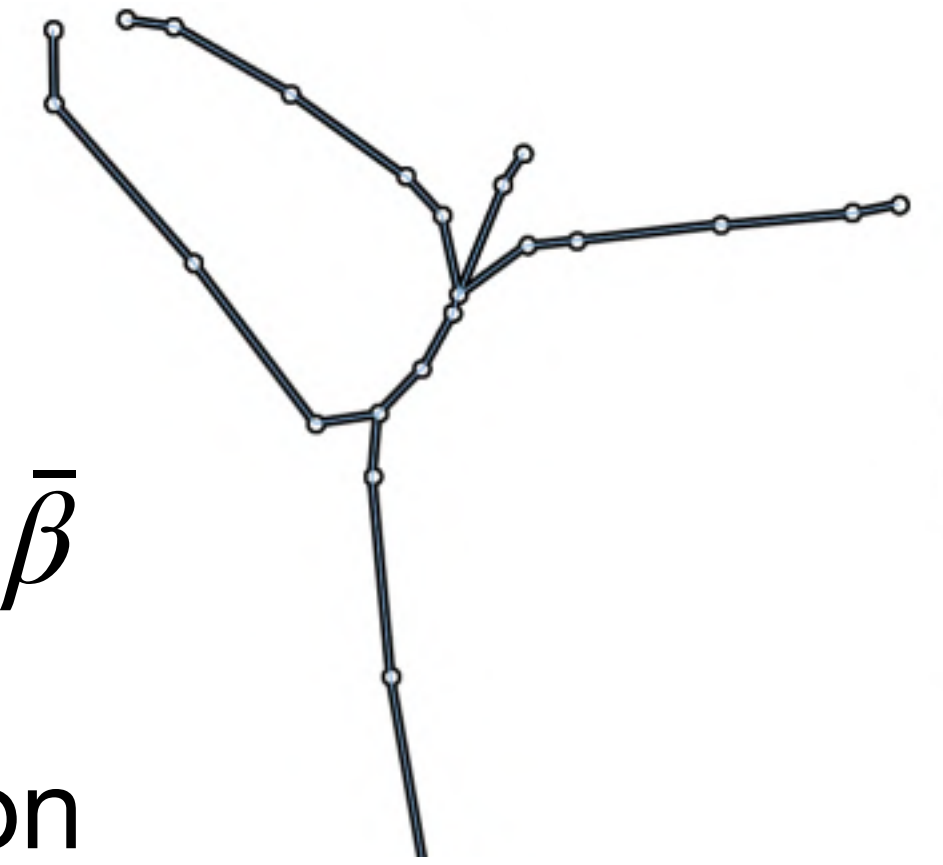
$$L_e(x_s, \omega_s)$$



$$L_o(x, \omega_o) = L_e(x, \omega_o) + \int_{\Omega} f(x, \omega_i, \omega_o) L_i(x, \omega_i) (\omega_i \cdot \mathbf{n}) d\omega_i$$

SMPL

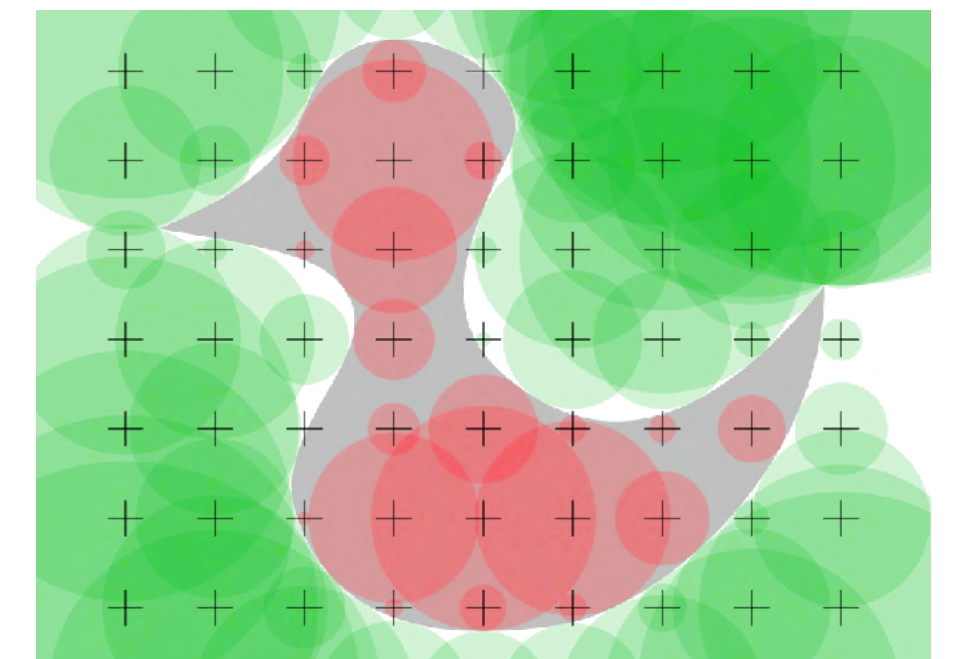
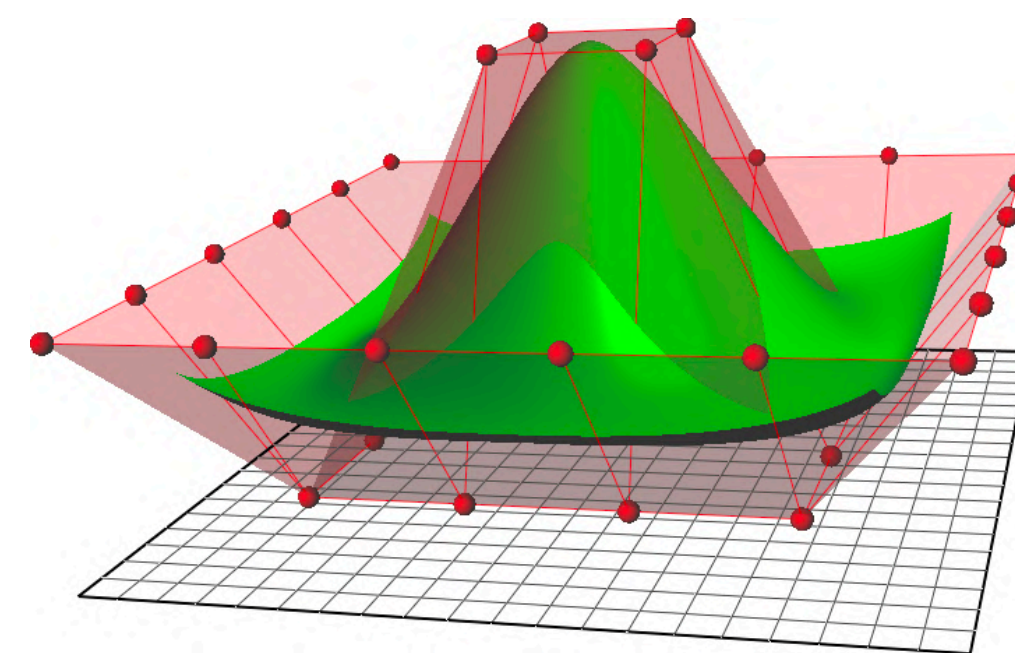
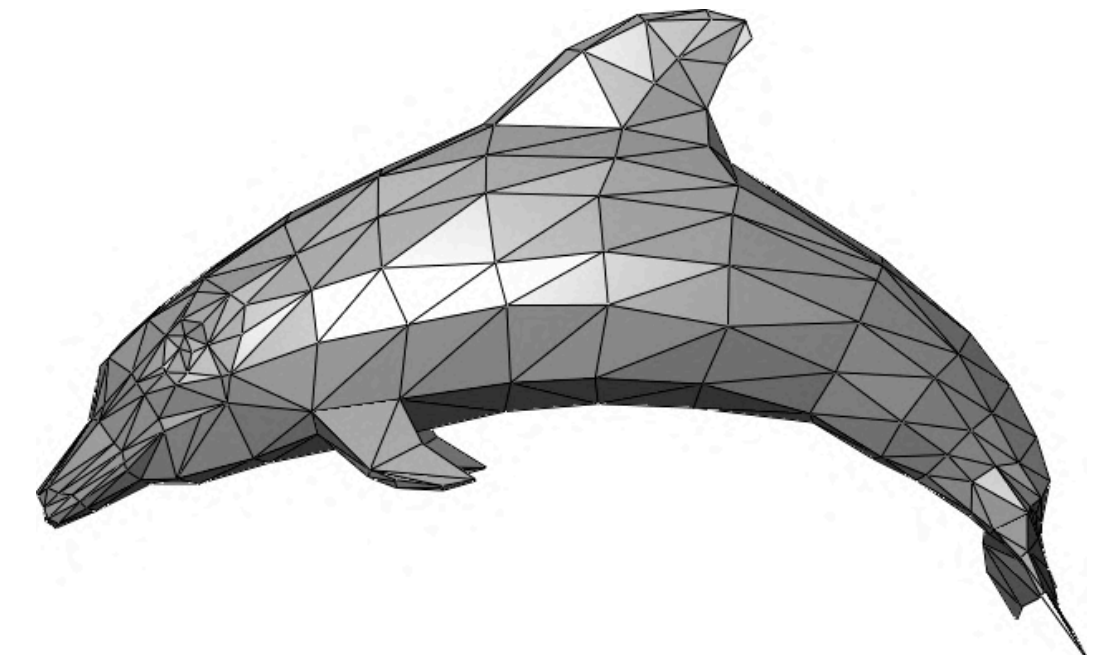
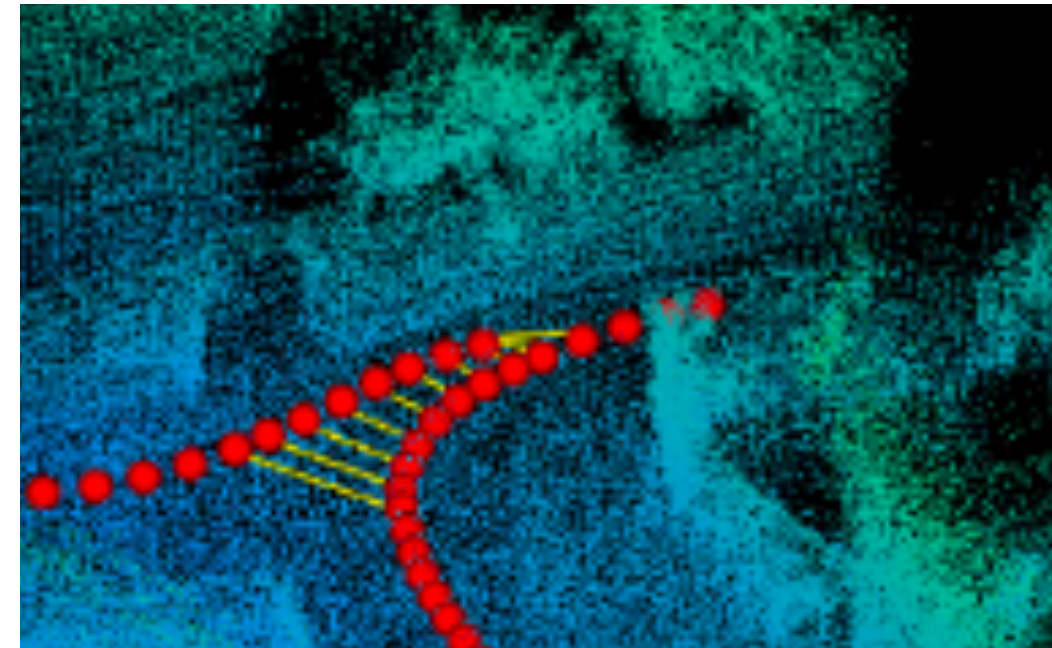
- Decompose arbitrary body into shape & pose
- PCA over human body shapes to parameterise natural diversity with $\vec{\beta}$
- Combine linear blend skinning ($\theta, J, \mathcal{W}$) & blend shapes for animation



Fields for 3D Representations

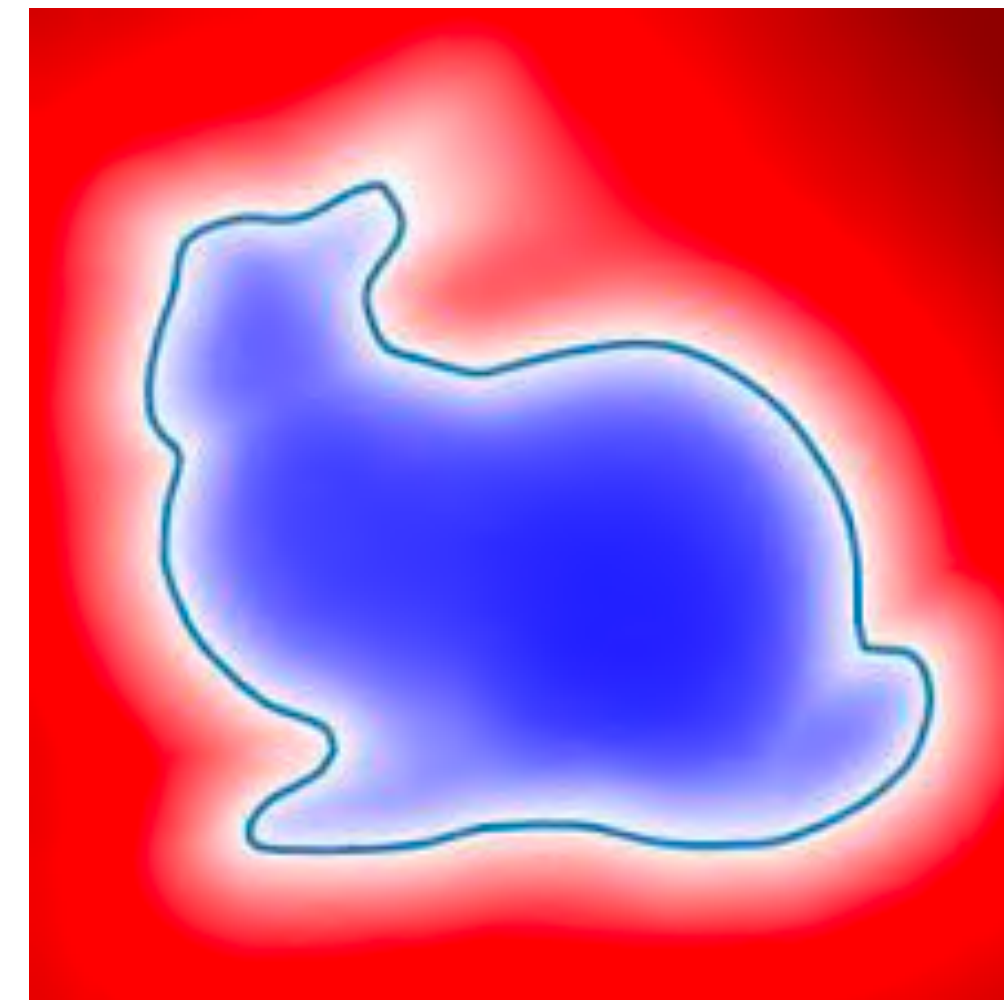
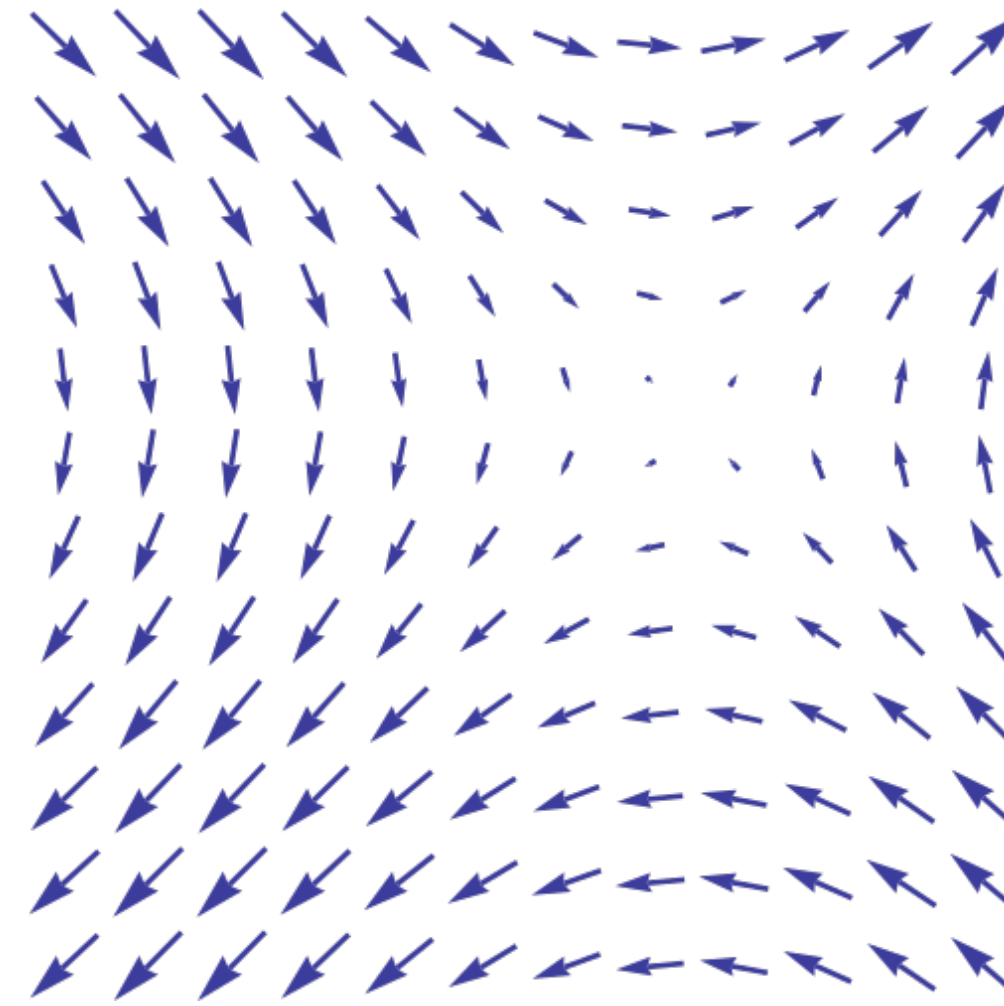
Common Representations

- Explicit
 - Point clouds
 - Polygonal meshes
 - Parametric $x = g(t)$
- Implicit
 - Isosurfaces $x : f(x) = 0$



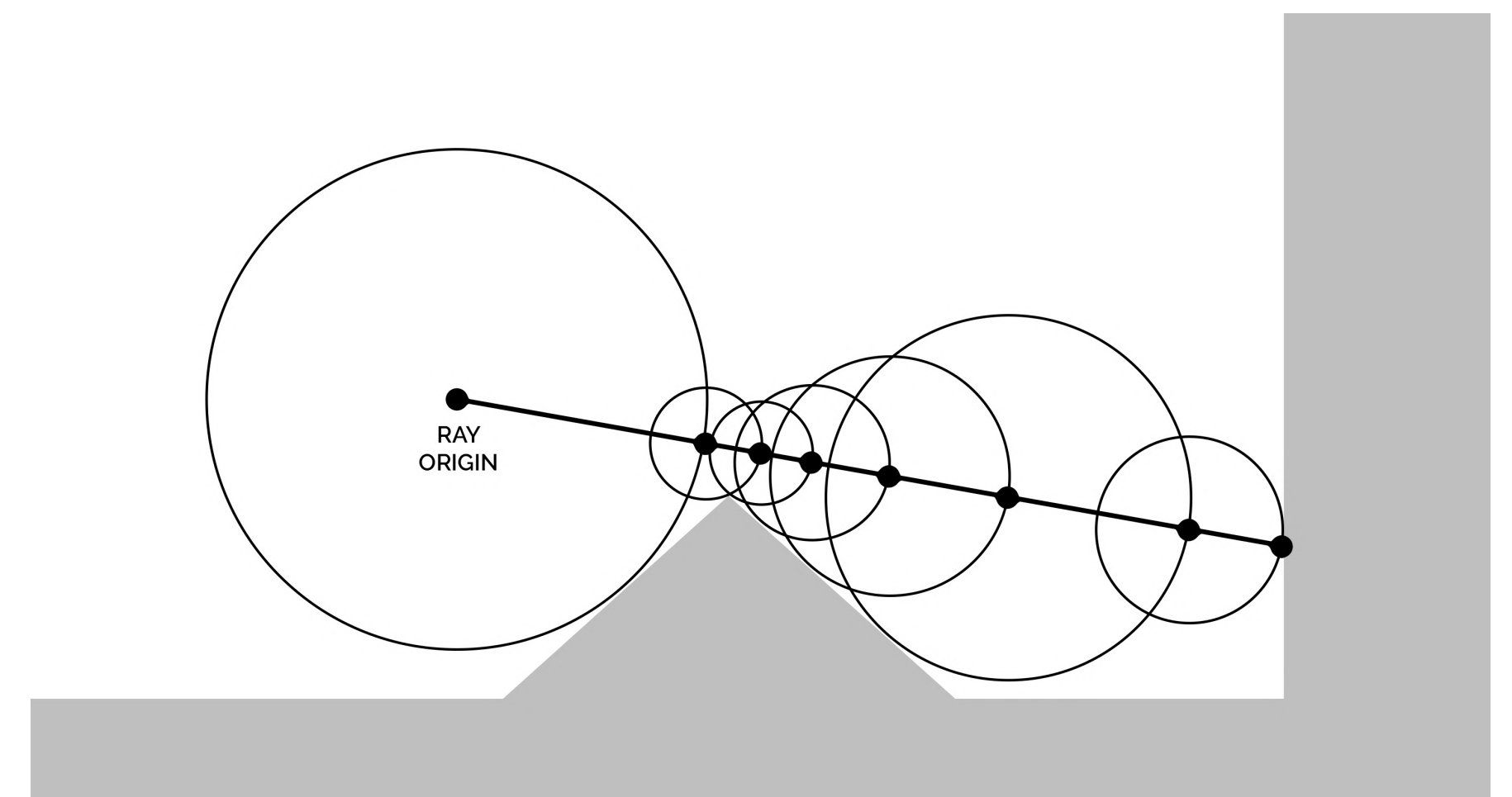
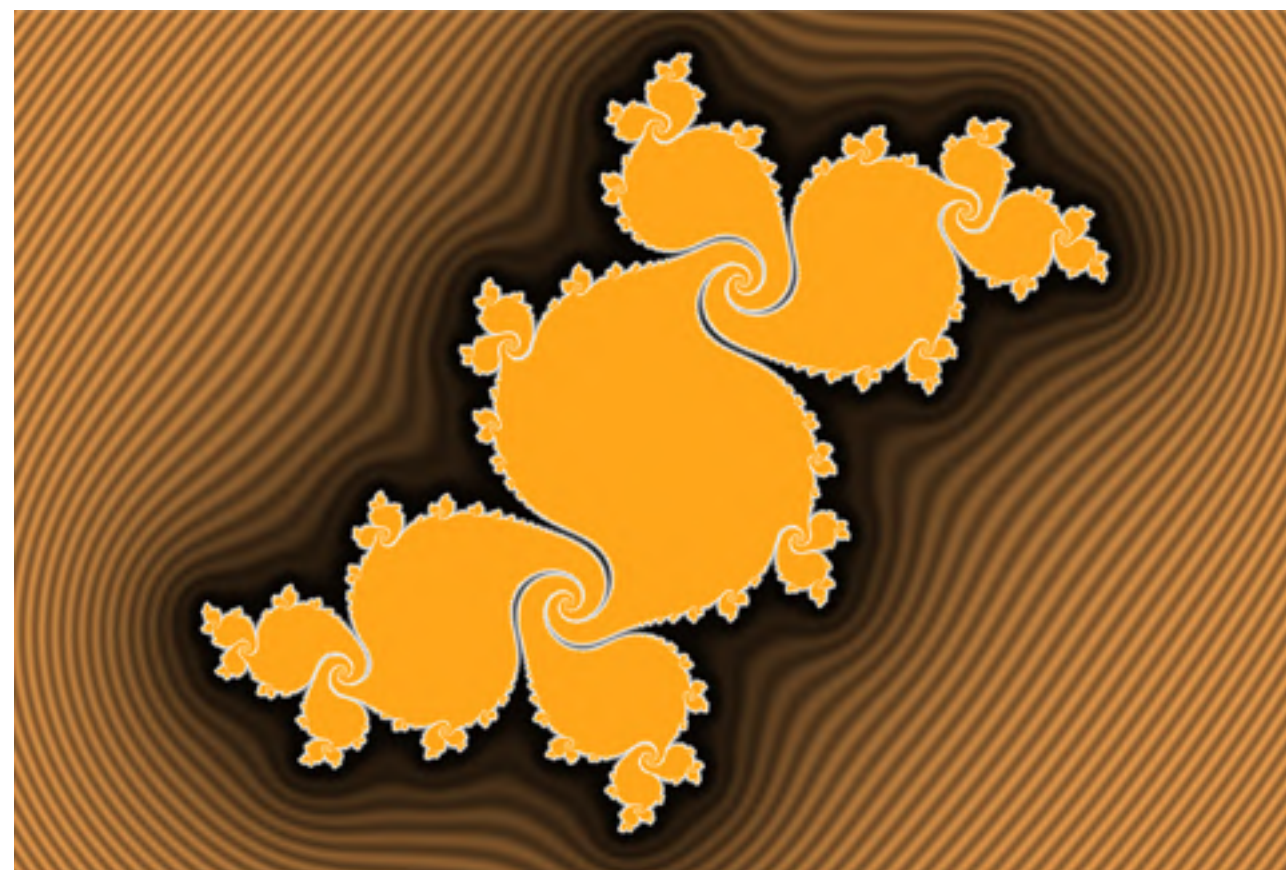
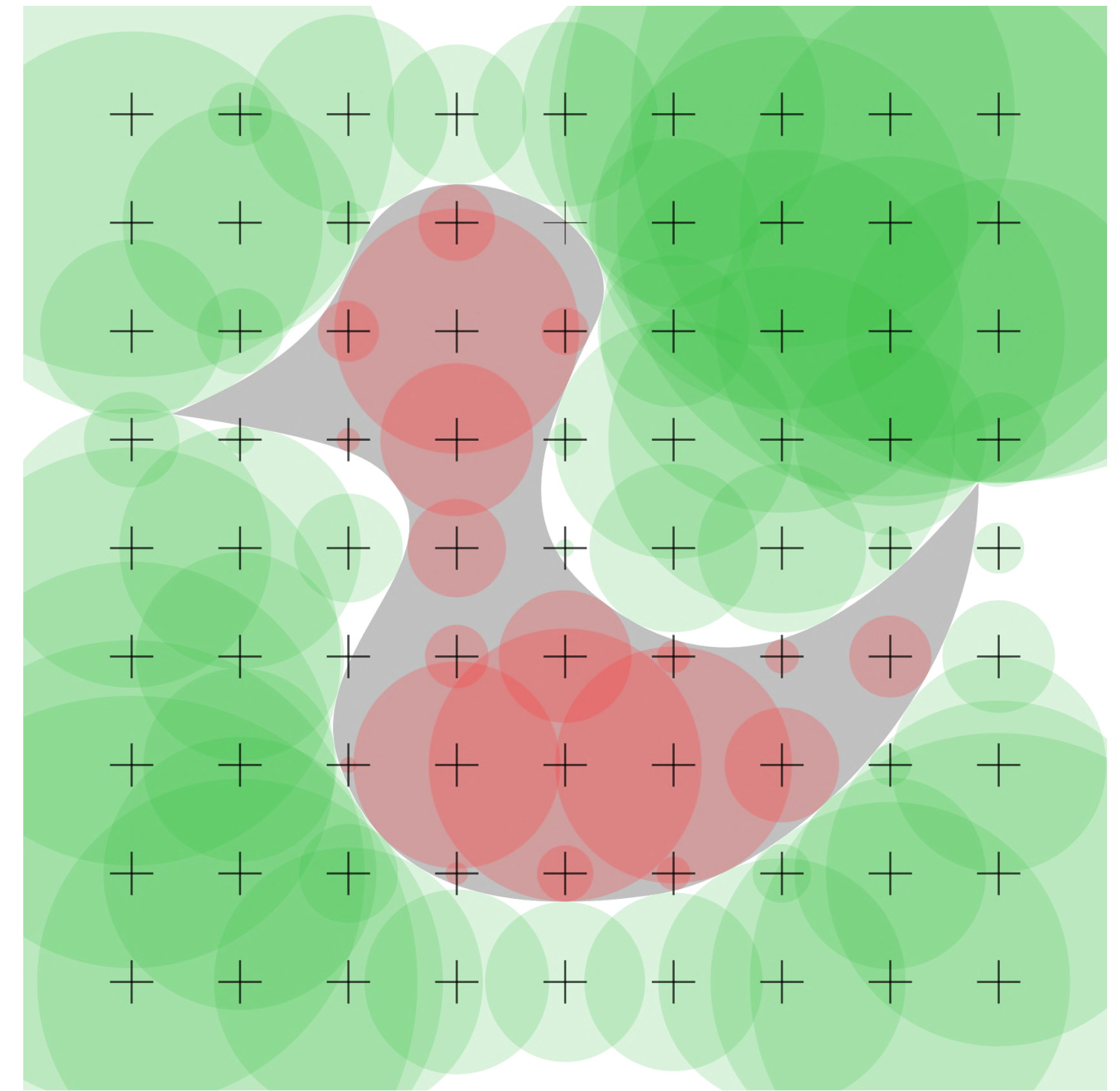
Vector Fields

- Used in physics and beyond
- Relevant examples
 - Light absorption coefficients
 - Signed Distance Fields
 - Occupancy Grids
 - Images



Signed Distance Fields

- Encode distance to the surface
- Great tool to parameterise primitive shapes & combinations
- Allow ray tracing with **sphere marching**



<https://iquilezles.org/articles/raymarchingdf/>

Volumetric Representations

- Surface is a common abstraction in computer graphics
 - Convenience, hardware optimization
 - Sometimes fail to represent real world phenomena
- As an alternative, we will represent scenes with “density” functions
 - Is there something in this point of space?
 - Does light pass through this point in space?
- **Scalar field:** each spatial point stores a scalar value

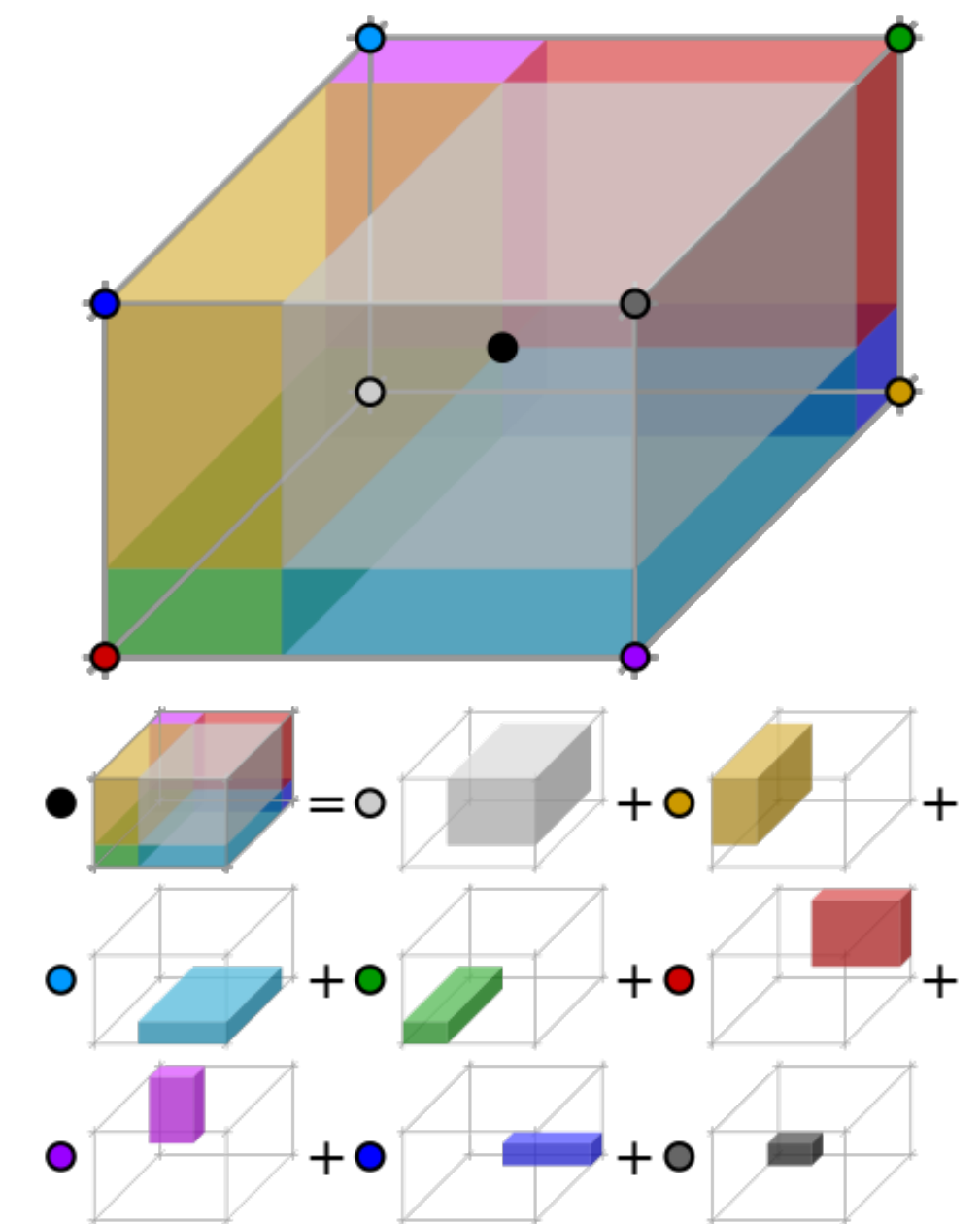
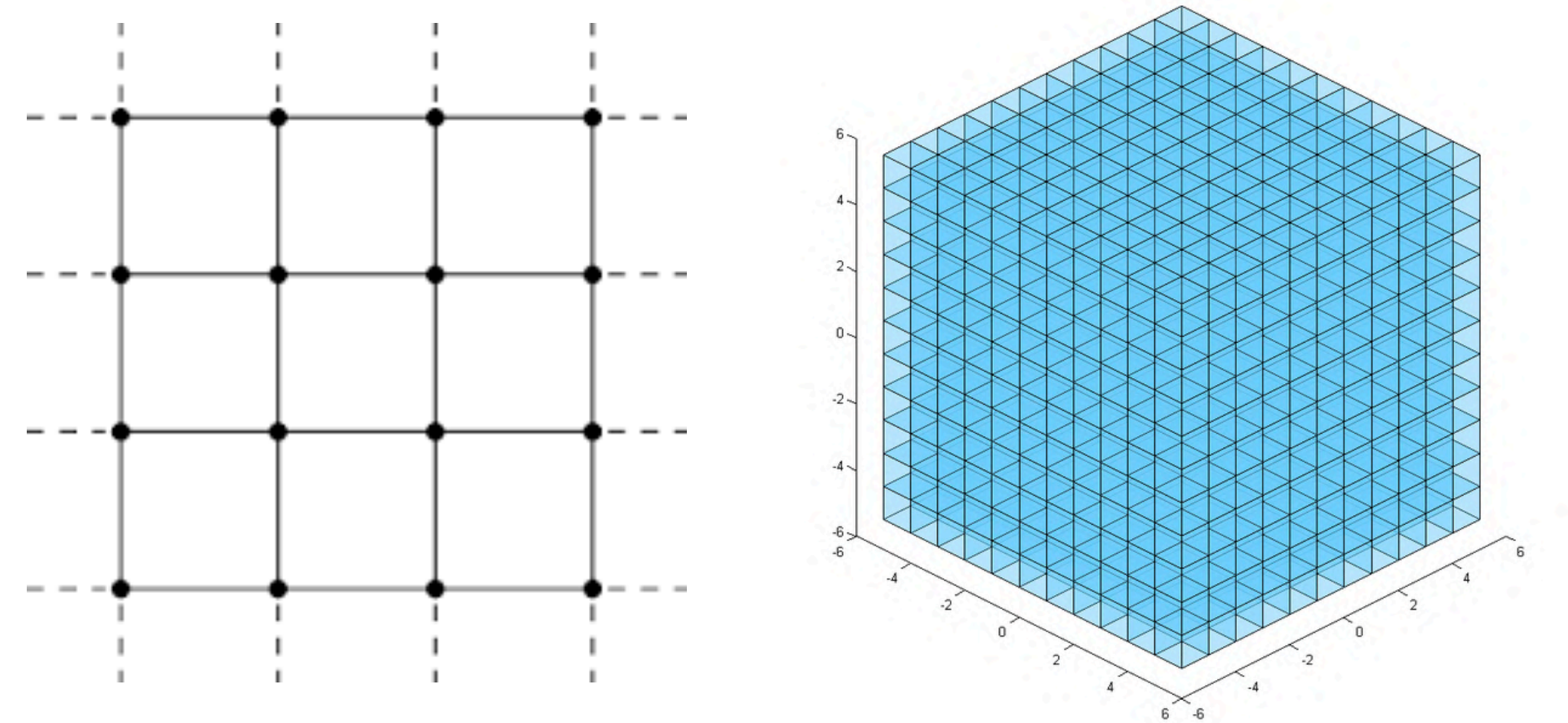


Grid Representations

Pixels, Voxels, etc.

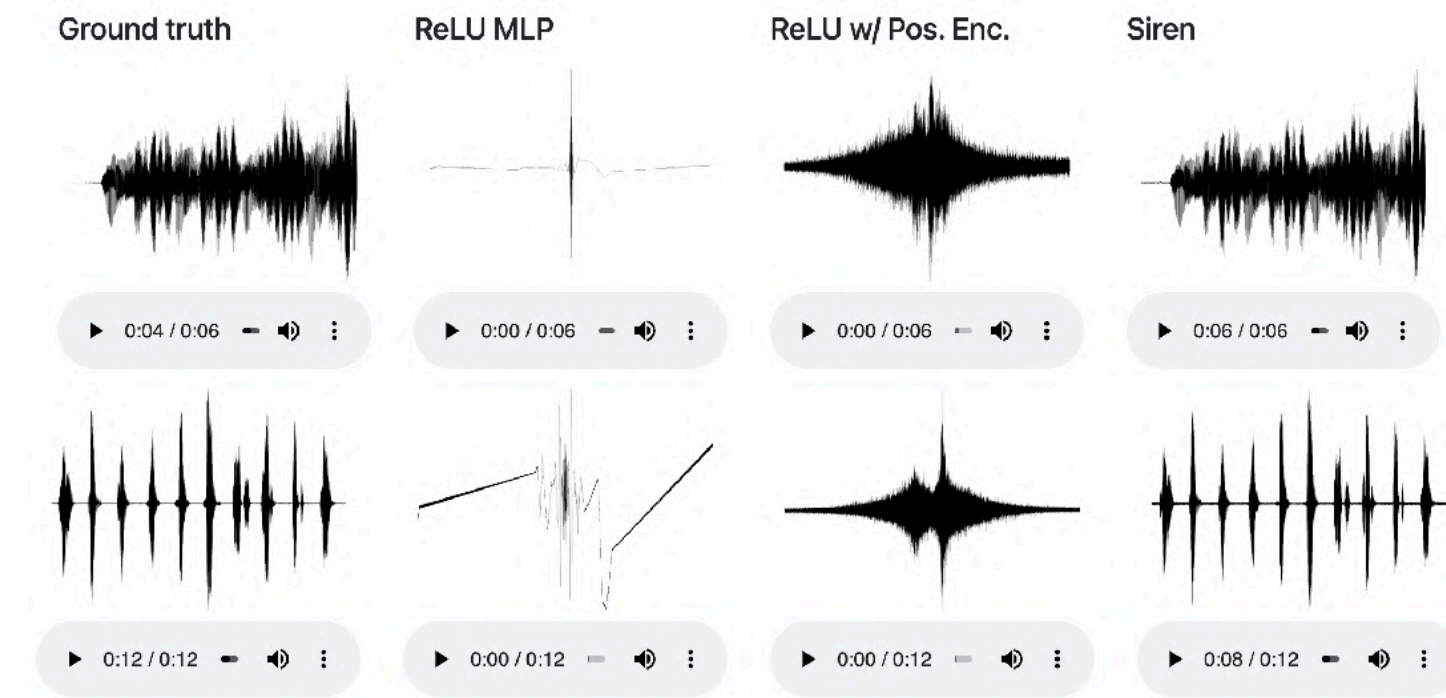
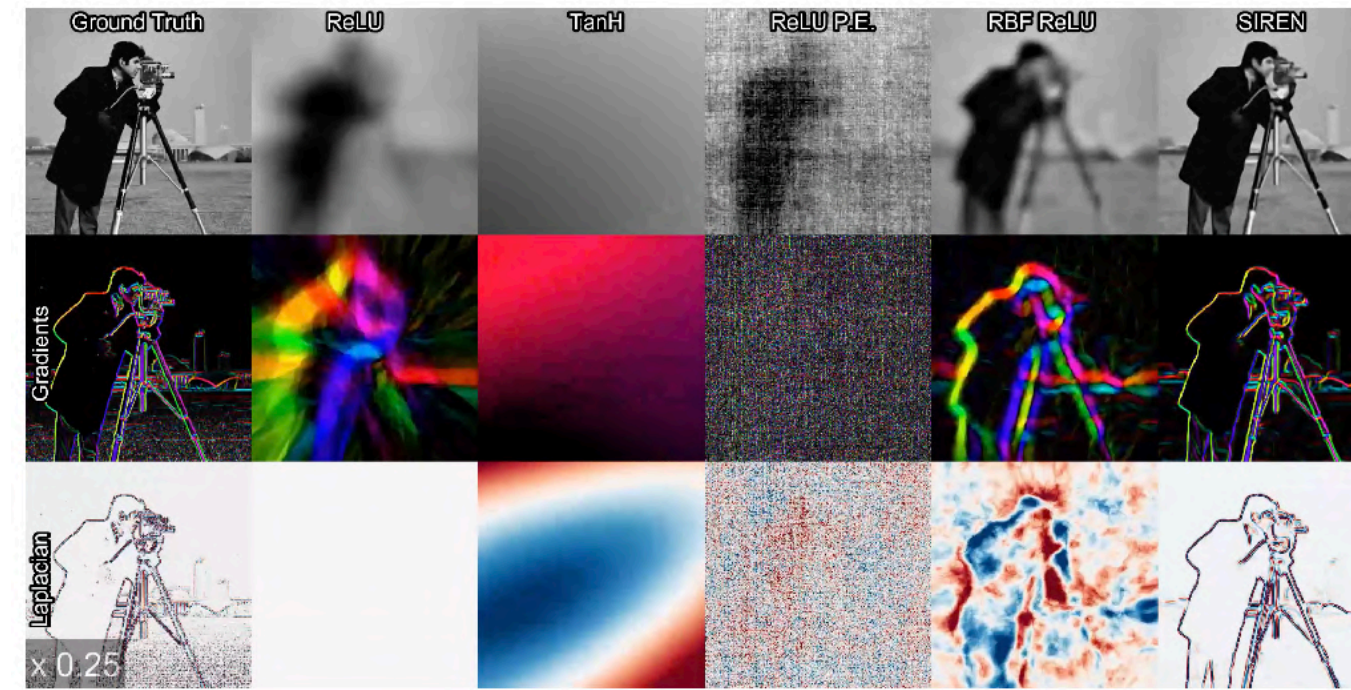
- Consider a field $f: [0,1]^3 \rightarrow \mathbb{R}^c$
- Define a $h \times w \times l$ grid in $[0,1]^3$
- Parameterize field with a tensor $G \in \mathbb{R}^{h \times w \times l \times c}$
- Interpolate G to define the field on $[0,1]^3$

$$f(x) = \sum_{i,j,k} G_{ijk} K(x, x_{ijk})$$



Neural Fields

- Takes a point x , returns a vector
- Parameterized MLP $f(\cdot)$
- Input encodings $\gamma(x)$ / trigonometric activations help
$$\gamma(x) = [\sin(2^k x), \cos(2^k x)]_{k=1}^h$$
- Applies to various domains such as photo, video, audio & 3D data



Hybrid Representations

Pros and cons of the above solutions

Is there a solution compromising between the two?

MLP

- Size: ~5mb
- Slow
- Trade-off speed for fidelity

Voxel Grid

- Size: ~1Gb
- Fast
- Trade-off memory for fidelity

Hybrid Representations

Voxel Based

- Parameterize a grid with $G \in \mathbb{R}^{h \times w \times l \times c}$
- For an input point x get the representation with interpolation

$$g(x) = \sum_{i,j,k} G_{ijk} K(x, x_{ijk})$$

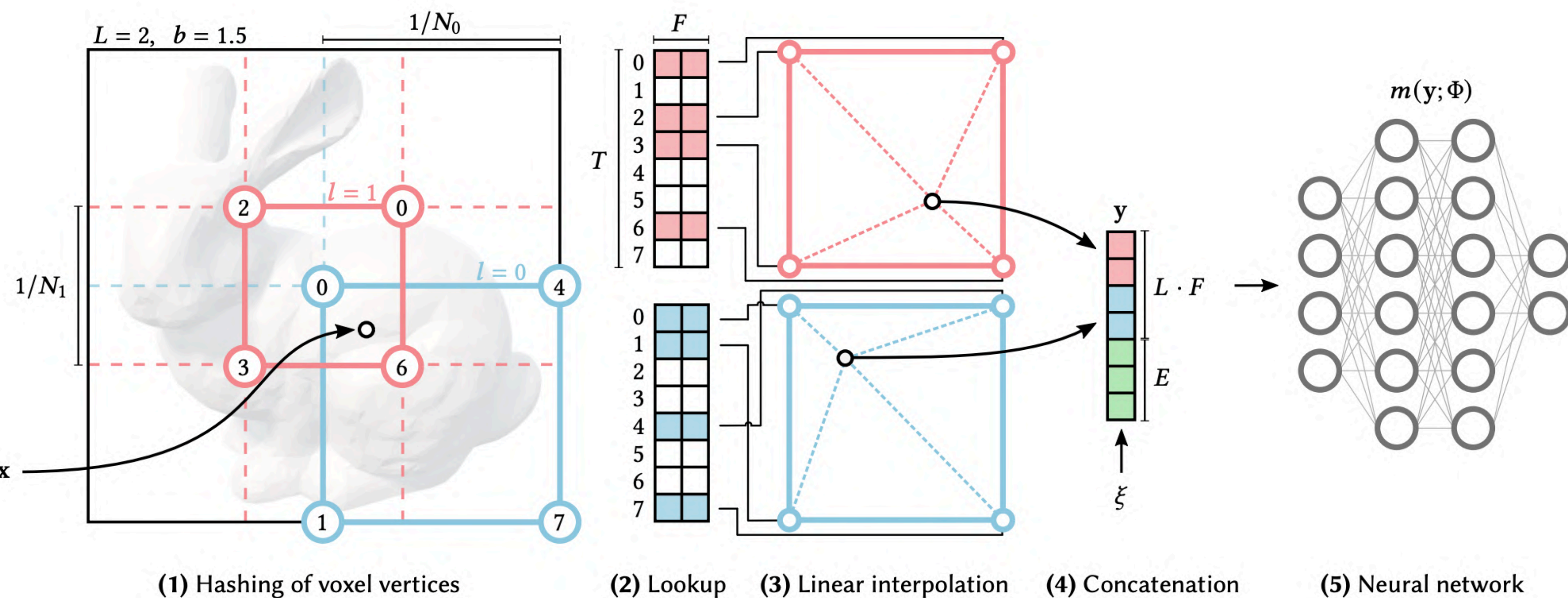
- Define an MLP $h(y)$ to map the representation to field vectors

$$f(x) := h(g(x))$$

- Middle ground between MLPs and voxel grids

Instant-NGP

Saving time and memory with HashGrids



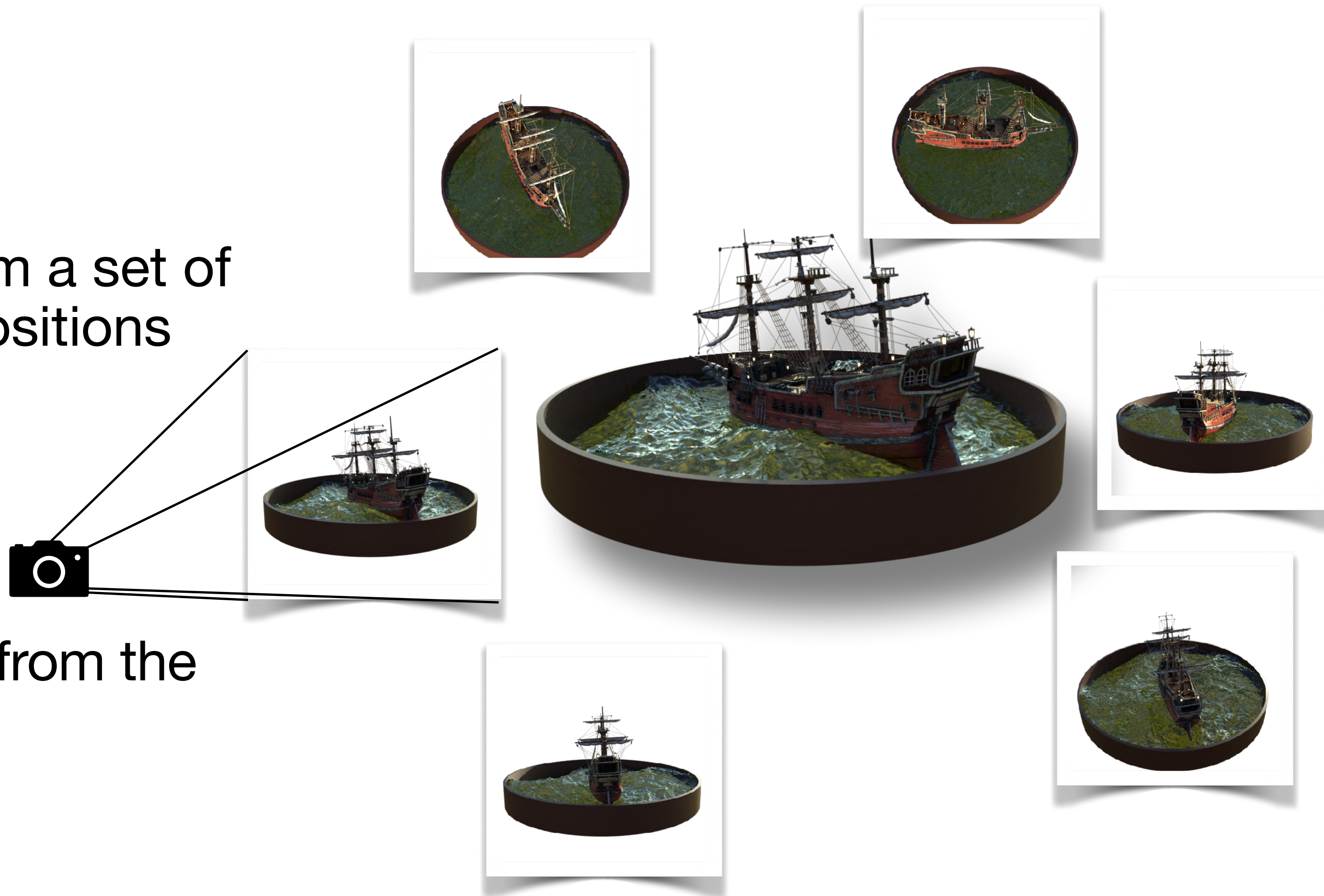
HashGrid

- Size ~100mb
- Average speed
- Flexibility factors
- HashGrid size
- MLP component

Novel View Synthesis with Neural Radiance Fields

Novel View Synthesis

- Problem setup:
 - A set of pictures taken from a set of pre-determined camera positions
 - Test camera position
- Goal:
 - A picture of a scene taken from the test camera position



Where Does the Data Come From?

- Structure From Motion (COLMAP)
 - Takes unposed images as input
- ARKit
 - Records device trajectory along with images



Neural Radiance Fields

Representing a Scene with Neural Fields

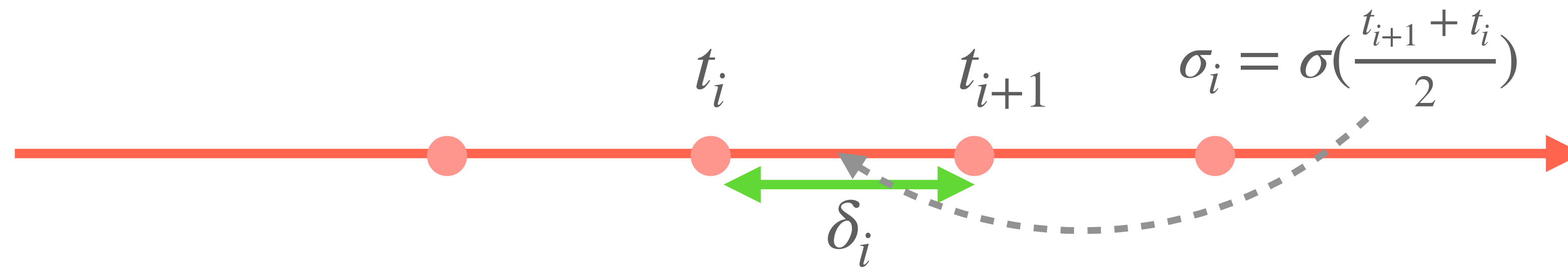
- Represent a scene with two fields
 - Density: $\sigma(x) : \mathbb{R}^3 \rightarrow \mathbb{R}^+$
 - Radiance: $C(x, d) : \mathbb{R}^3 \times S^2 \rightarrow \mathbb{R}^3$
- Density represents is related to opacity
- Radiance represents color of a point
- Architecture: MLP + positional embeddings



Computing Pixel Color

- Density $\sigma(t) \in [0, +\infty)$ is related to opacity at t
- Divide the ray with points $t_1, \dots, t_n$ and define

$$\alpha_i = 1 - \exp(-\sigma_i \delta_i)$$



- Expected color along a ray is given by

$$C = \sum_i c(t_i) \cdot \alpha_i \prod_{j < i} (1 - \alpha_j)$$

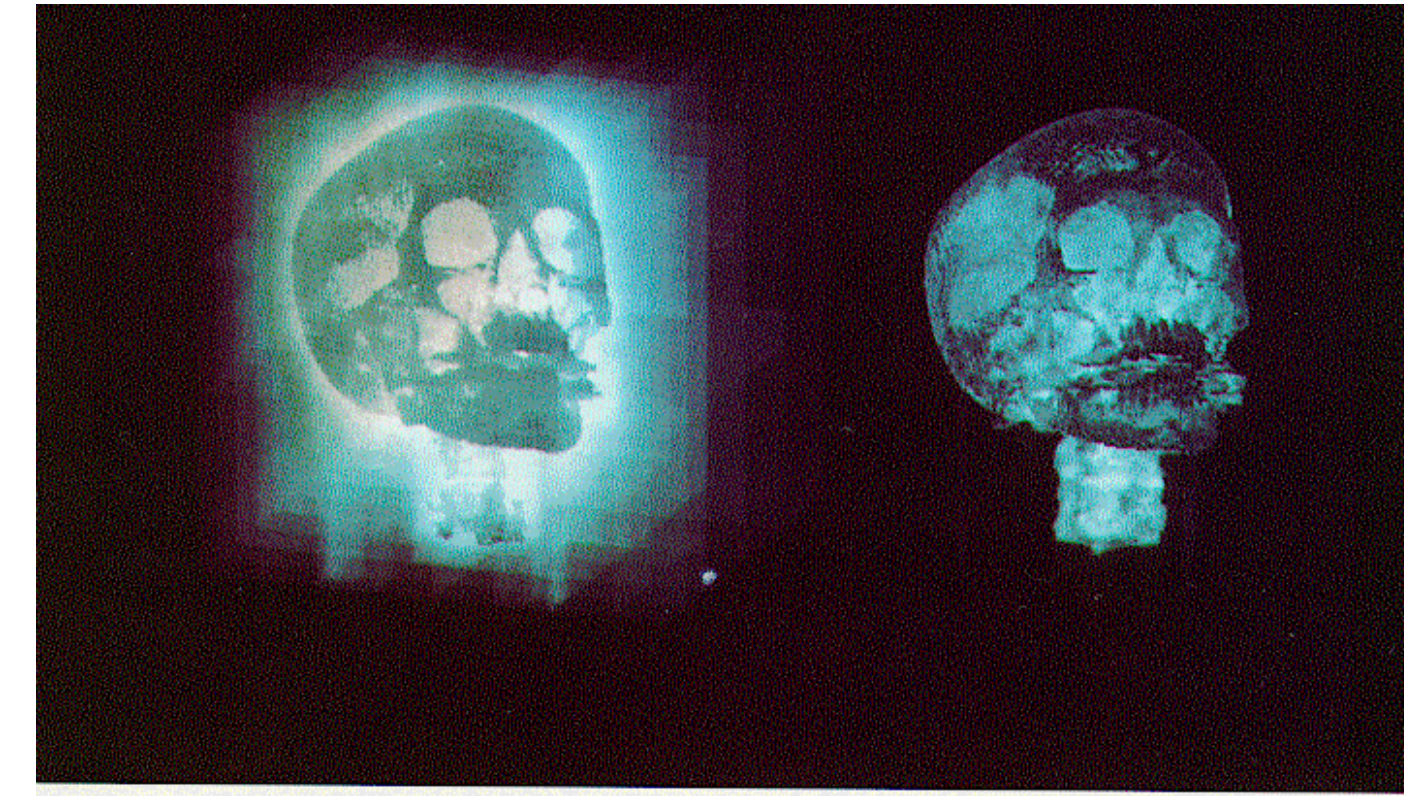


Fig. 11. Costs of rendering Figure 8 using hierarchical enumeration and adaptive termination.

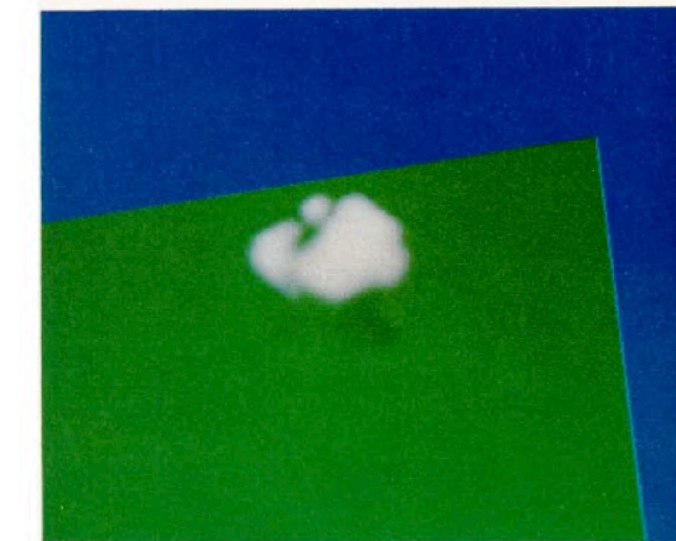


Fig. 5

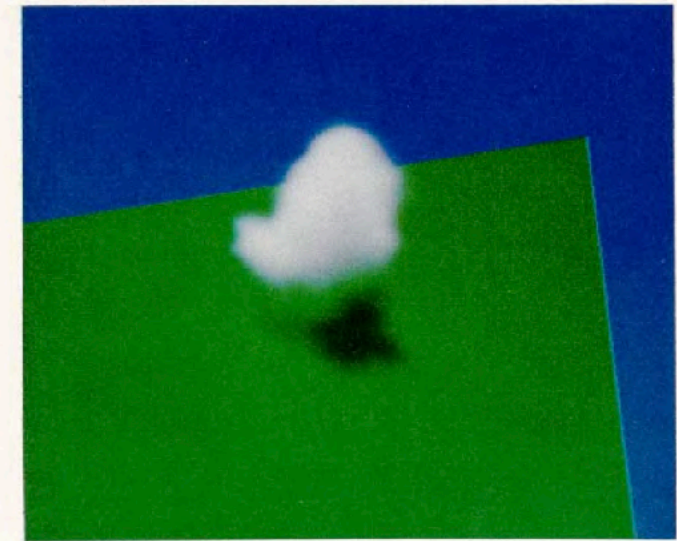


Fig. 8

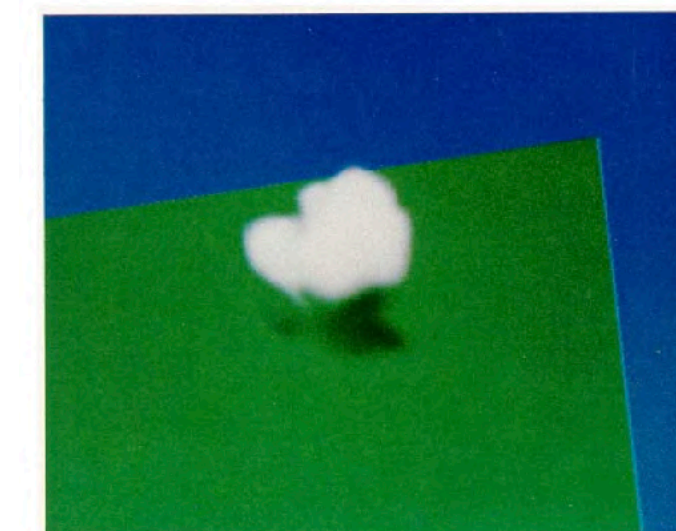


Fig. 6

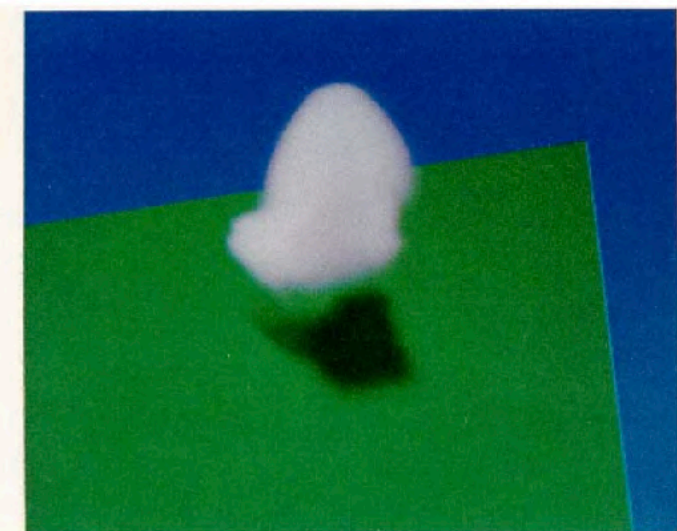


Fig. 9

Neural Radiance Fields

Optimisation

- Training: minimise the mean squared error $\mathbb{E}_D \|C - C_{gt}\|^2$

$$C = \sum_i c_i \alpha_i \prod_{j < i} (1 - \alpha_j)$$

- Optimise w.r.t. parameters of radiance C and density σ

Training Visualisation

- Synthetic data

Course Iters.: 1
Eps. time: 00:00

Course Iters.: 1
Eps. time: 00:00

Course Iters.: 1
Eps. time: 00:00

- Real forward-facing

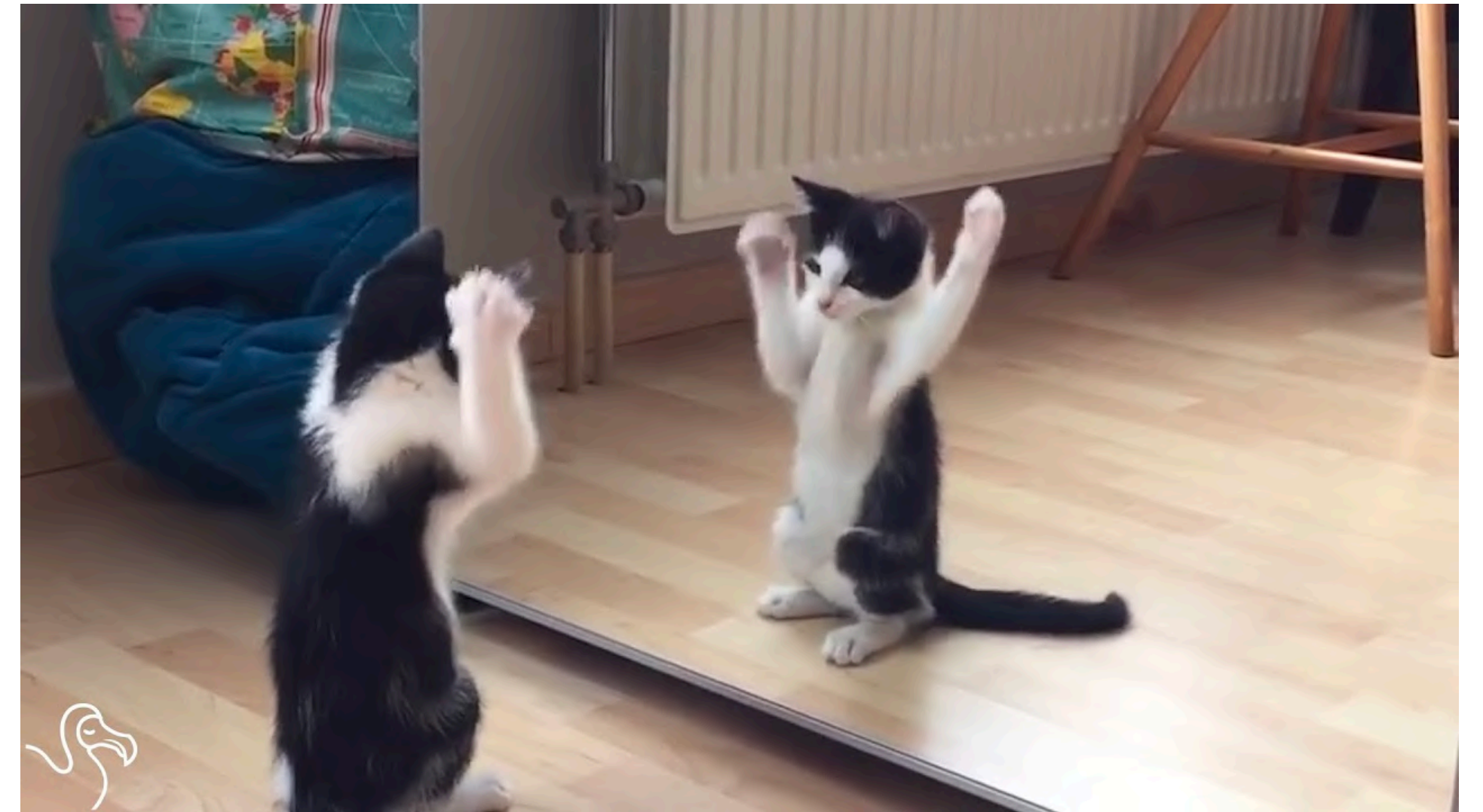
Iter.: 1
Eps. time: 00:00 (>> xD)

Iter.: 1
Eps. time: 00:00 (>> xD)

Iter.: 1
Eps. time: 00:00 (>> xD)

NeRF pros and cons

- + Simplicity & Flexibility
- + Photorealism
- + Compression (*5mb / scene*)
- Ill-posed problem
- Long training time (*48 h. / scene*)
- Slow rendering (*1 min. / frame*)

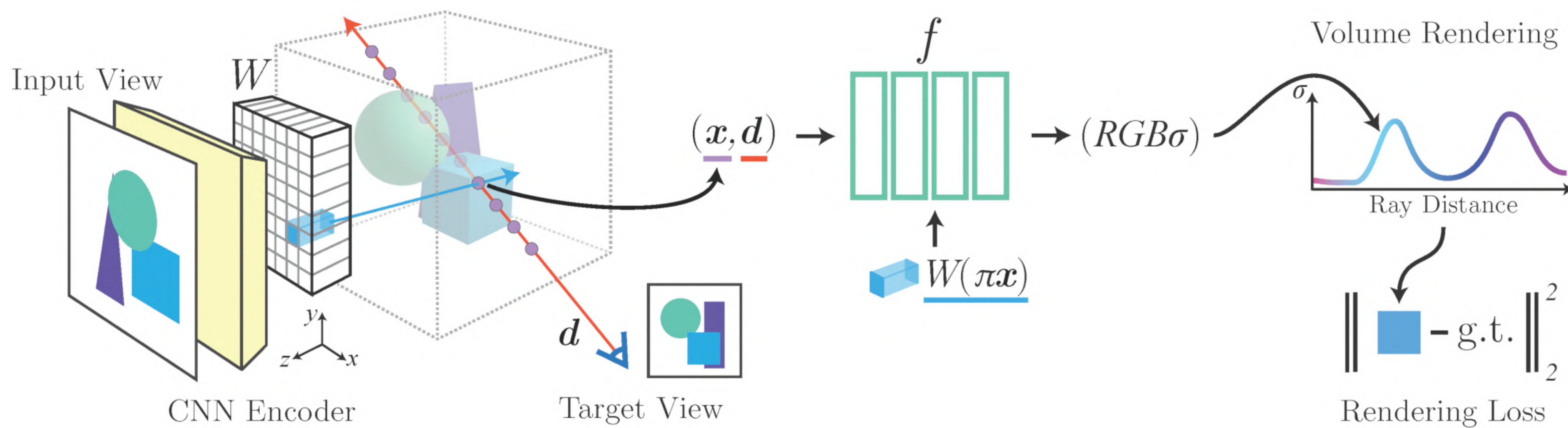


NeRF in The Wild & Block NeRF

- Radiance field handle embeddings capturing variability in lightning conditions
- Approach is scalable to large scene reconstruction



PixelNerf



Language Embedded Radiance Fields

- One can replace MSE reconstruction loss with fancier training signals
 - CLIP-embeddings for text retrieval and 3D segmentation



Improving Base Model: Time

Computational Overhead

$$\hat{C} = \sum_i c_i \alpha_i \prod_{j < i} (1 - \alpha_j)$$

- Picture contains $\approx 10^6$ pixels with 10^2 field evaluations per pixel
- Modern GPUs deliver poor frame rate at this cost
- How can we speed up rendering?



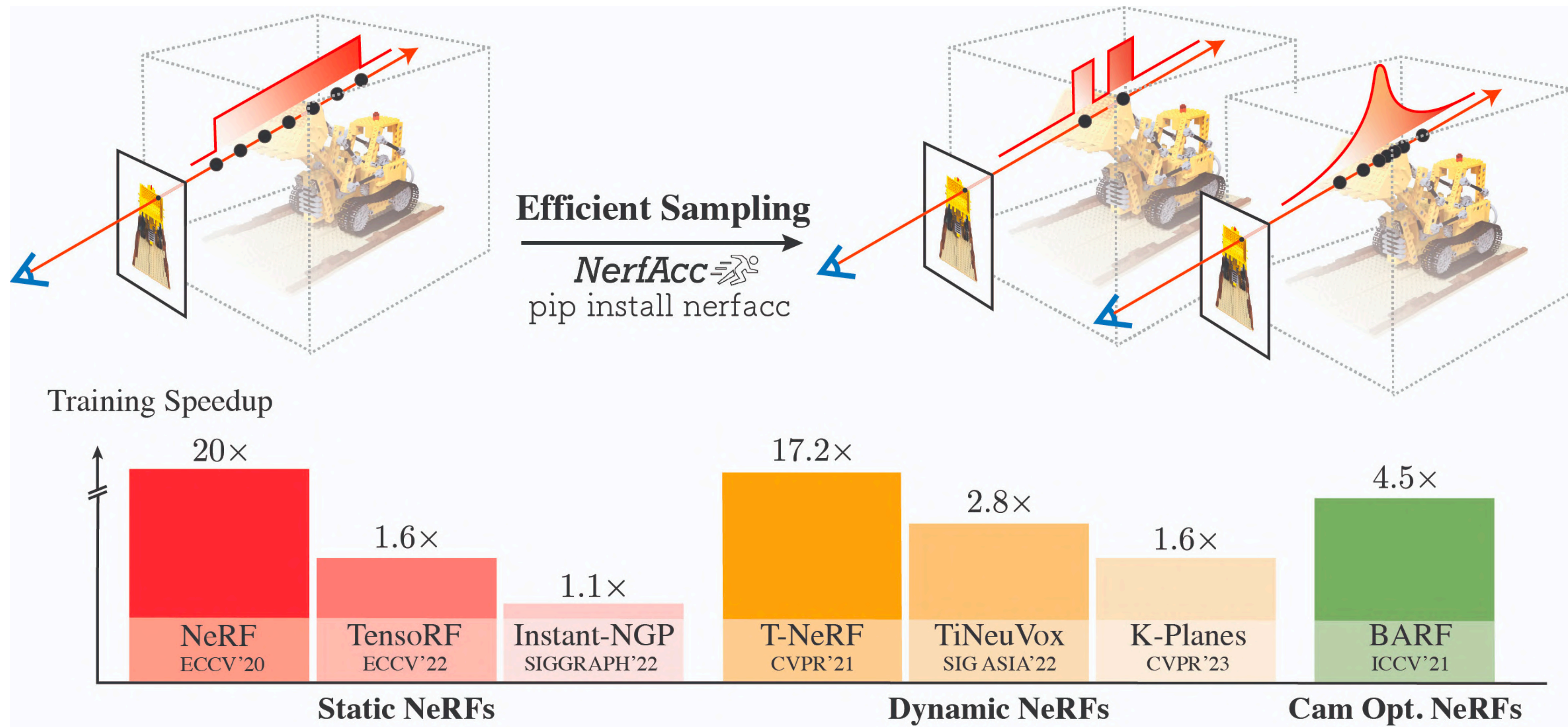
Sparsification

- Each terms in the sum requires running an MLP

$$C = \sum_i c_i \alpha_i \prod_{j < i} (1 - \alpha_j)$$

- If $\sigma(x_i) = 0$, then $1 - \alpha_i = 0$, we can omit i -th term
- Idea: cache $\sigma(\cdot)$, omit i if we know $\sigma(x_i) = 0$
- Outcome: training and rendering 10-30 times faster

NerfAcc

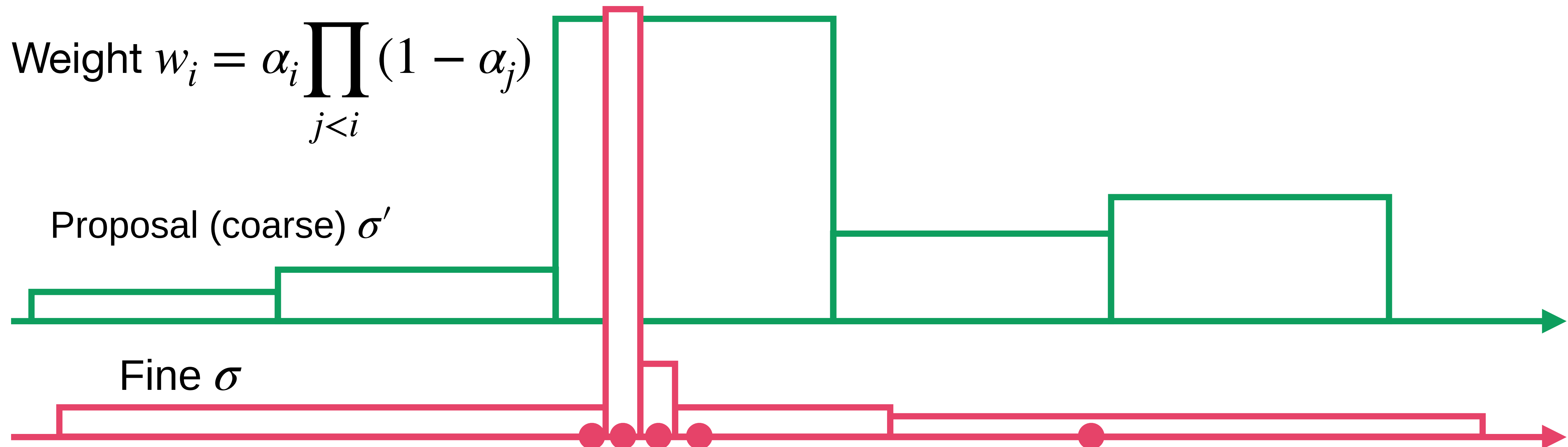


Improving Base Model: Fidelity

Hierarchical Sampling (3)

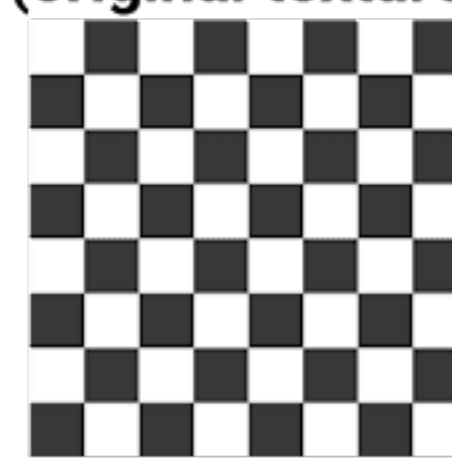
- Until now, we picked m knots uniformly on the ray
- Hierarchical approach picks the knots adaptively

- Weight $w_i = \alpha_i \prod_{j < i} (1 - \alpha_j)$



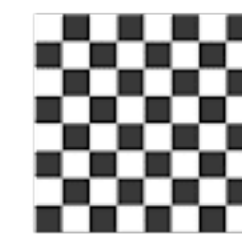
Aliasing

Mip 0
(original texture)



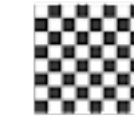
128 x 128

Mip 1



64 x 64

Mip 2



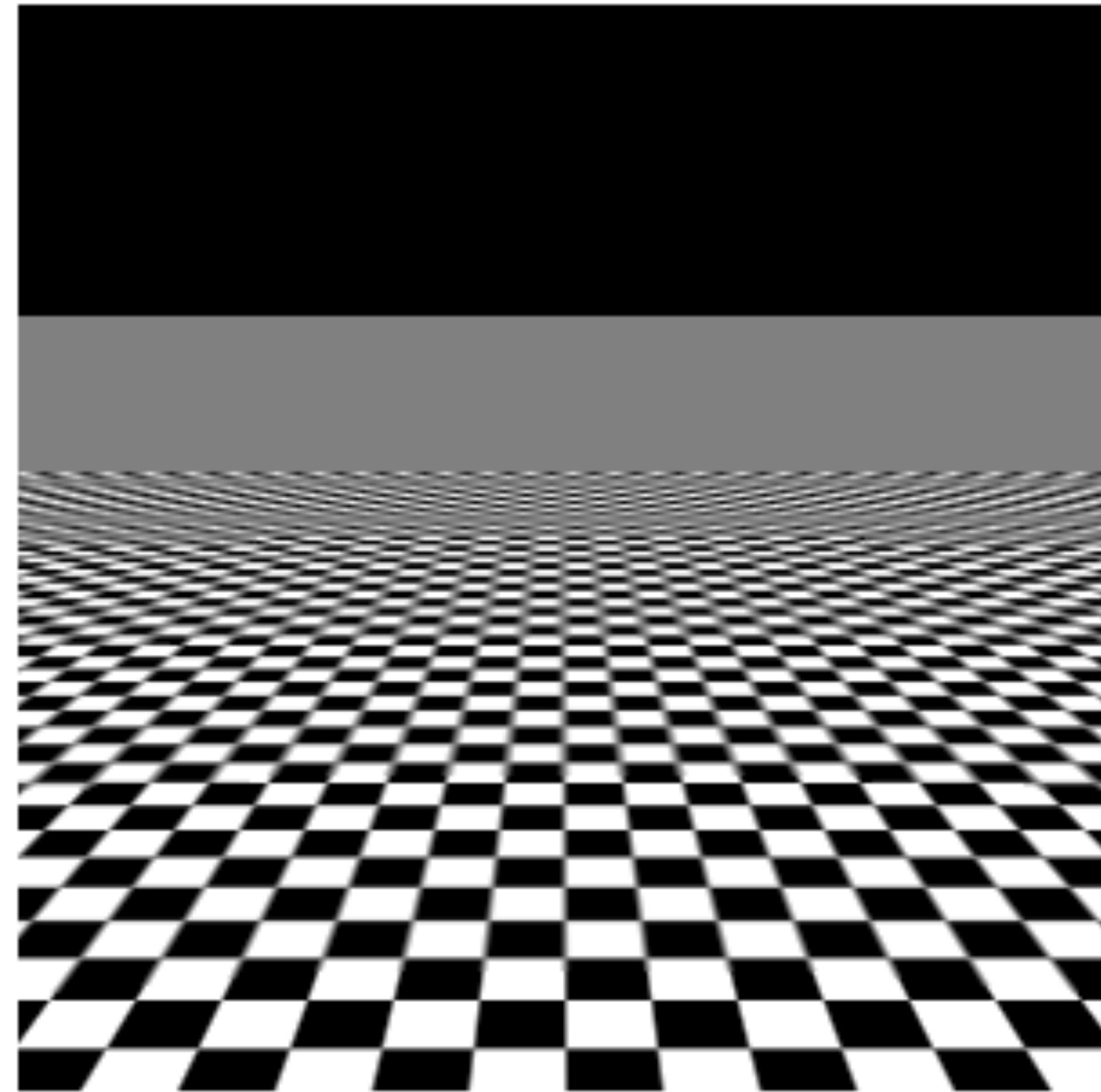
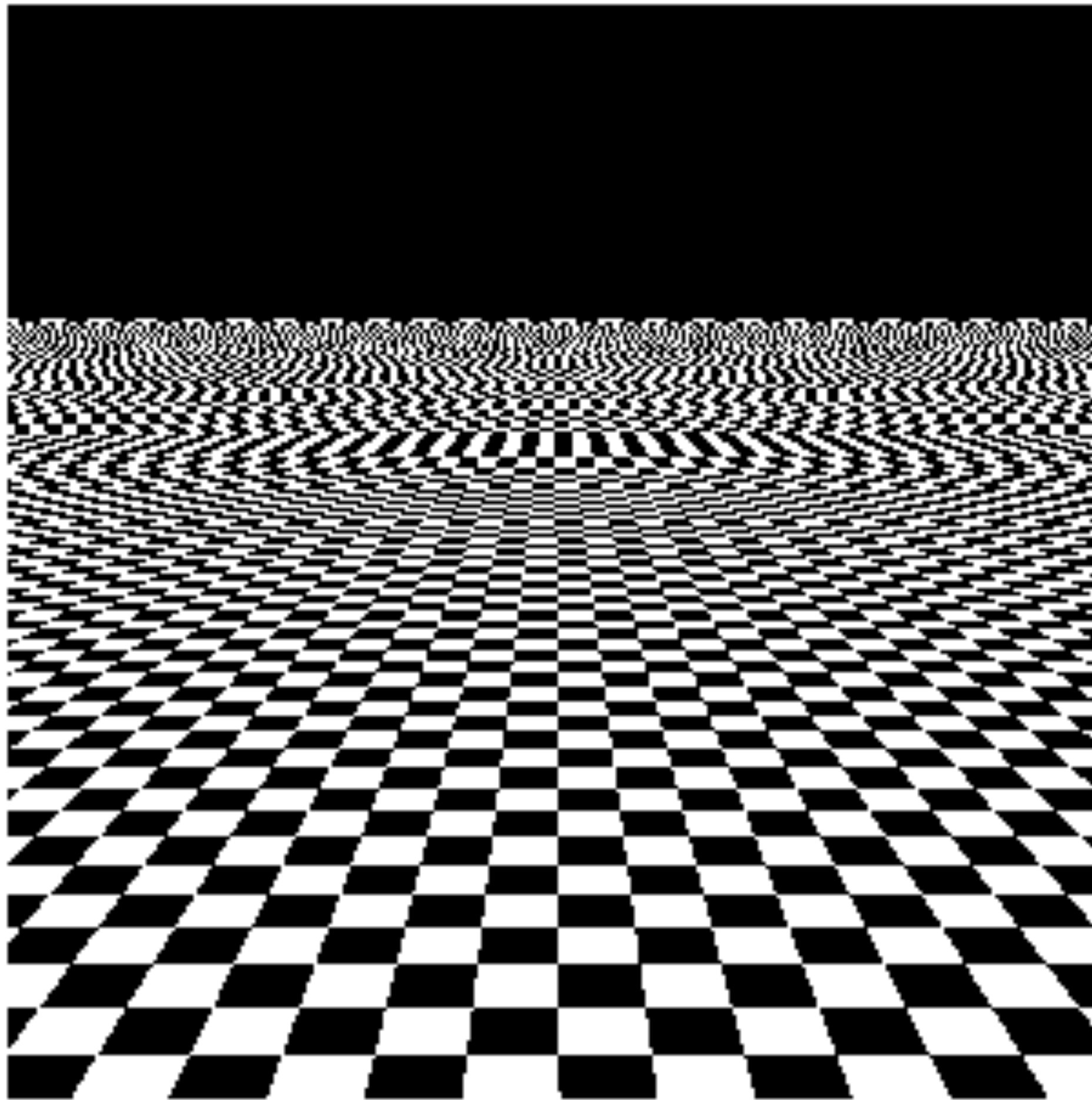
32 x 32

Mip 3



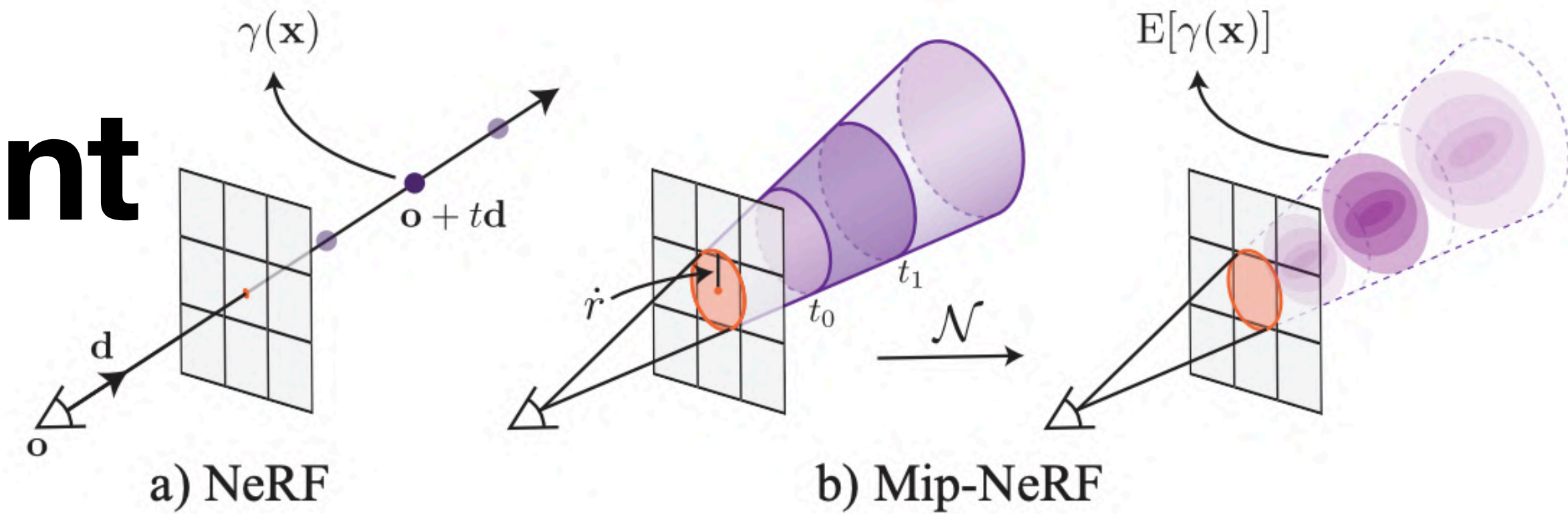
16 x 16

multum in parvo



Taking Scale into Account

A Parallel Line of Research



- Original NeRF experiments had biased data
 - Camera distances did not change by a significant amount
- Pixels cover a cone-like region in space
- Ideal solution - compute the average radiance over the whole cone
- Mip-NeRF approximates the average radiance/density over a cone segment
- Idea: compute embedding average rather than output average

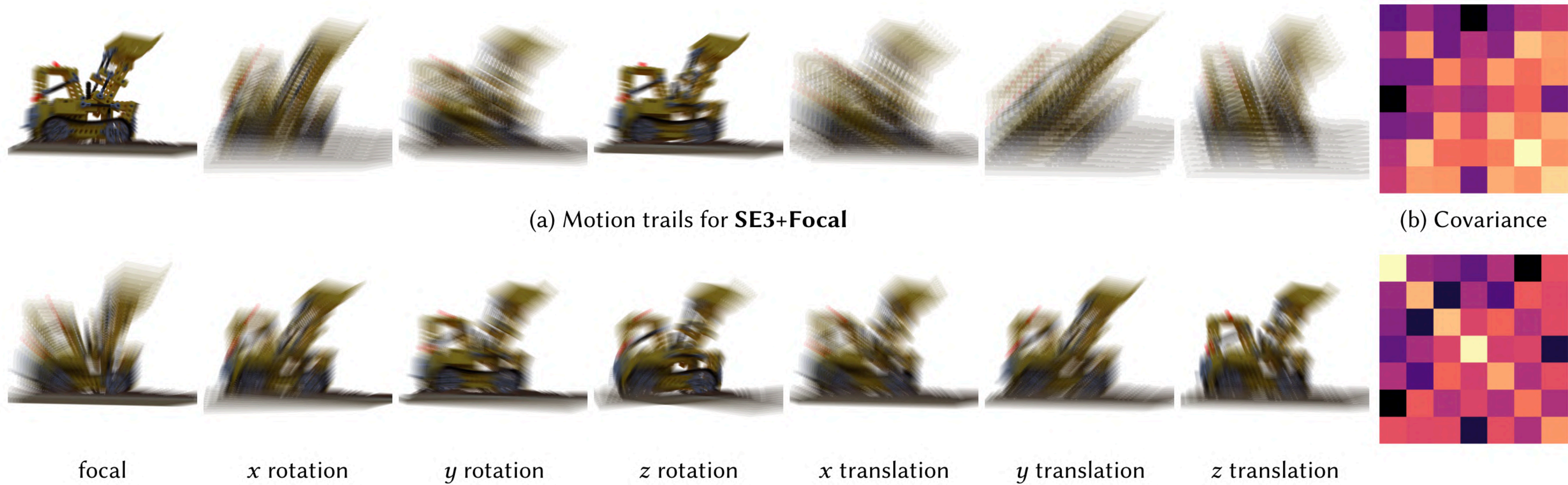


Zip-NeRF

- Merges two research branches
 - Speed: Instant-NGP
 - Fidelity: Mip-NeRF-360
- Mip-NeRF is designed for MLPs
- Zip-NeRF adapts Mip-maps to Instant-NGP



Fine-tuning Camera Positions





Improving Scene Geometry

Novel View Synthesis

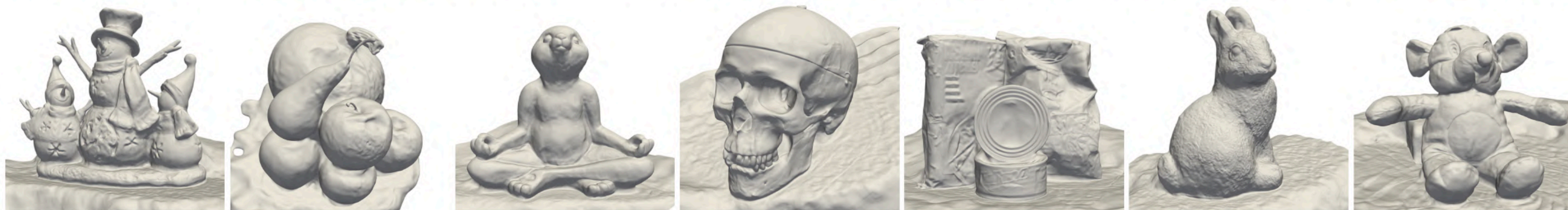
Colmap



NeRF



VoISDF



Combining Density Fields and SDFs

- Let d_{Ω} be the signed distance field for shape Ω
- Parameterise density as $\sigma(x) = \alpha \Phi \left(-\frac{d_{\Omega}(x)}{\beta} \right)$
- Where $\Phi(x)$ is the Laplace distribution CDF

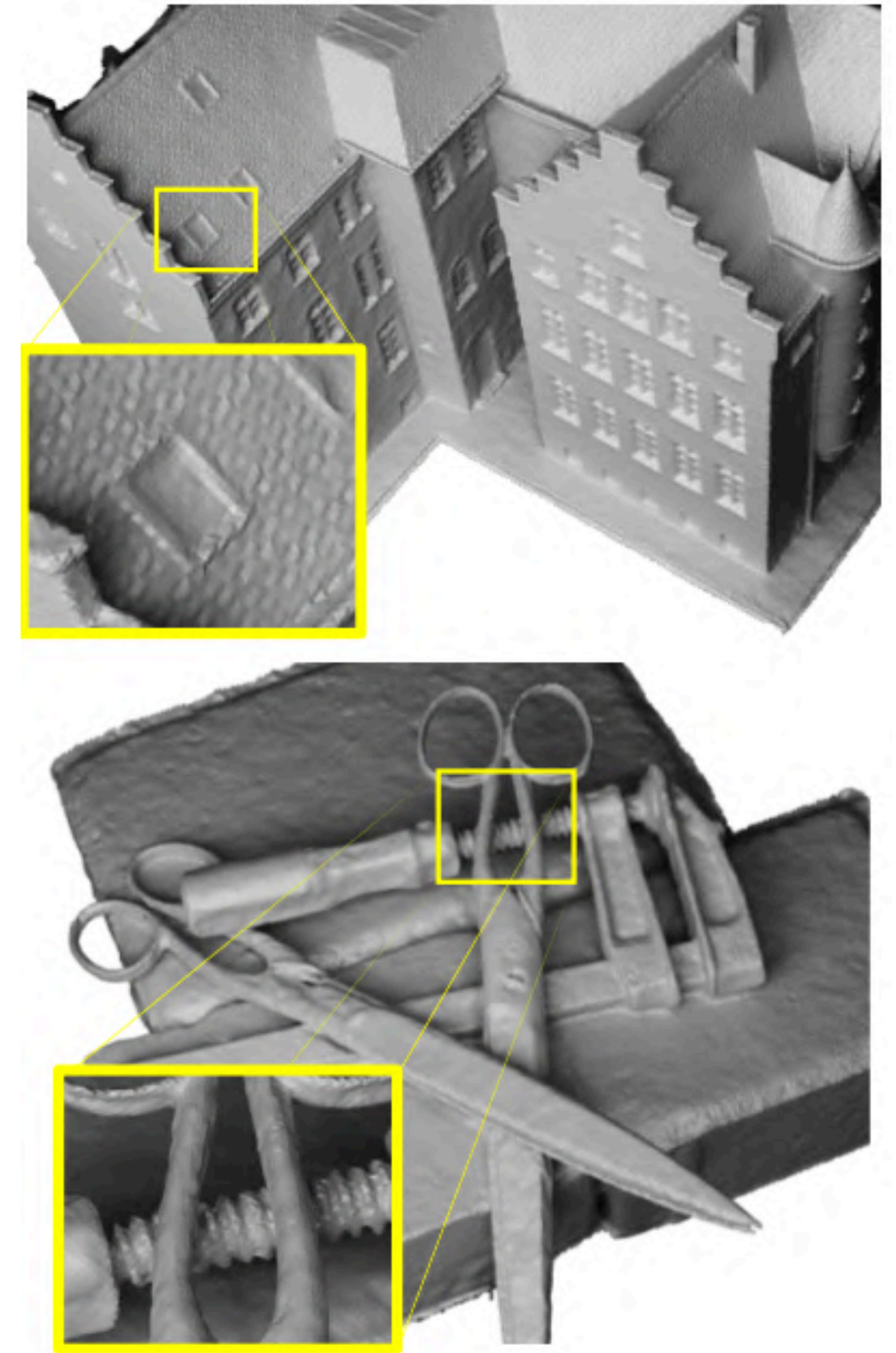


$$\Psi(s) = \begin{cases} \frac{1}{2} \exp(s) & \text{if } s \leq 0 \\ 1 - \frac{1}{2} \exp(-s) & \text{if } s > 0 \end{cases}$$

Eikonal Equation

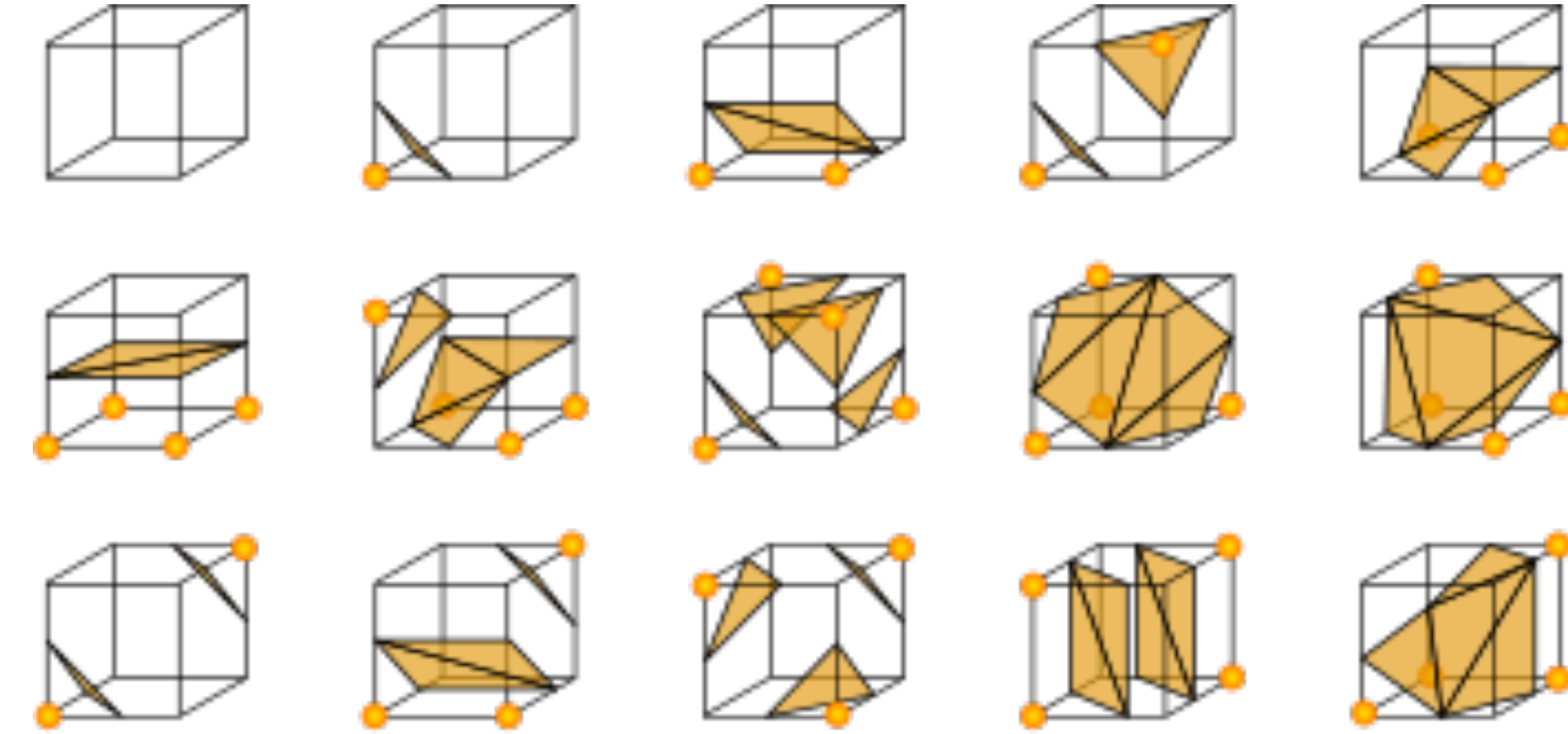
- How does the gradient of SDF look like?
- The gradient $\nabla d_{\Omega}(x)$ points away from the closest surface point
- The norm of the gradient is constant:
 $\|\nabla d_{\Omega}(x)\| = 1$
- Eikonal loss:

$$\mathcal{L}_{SDF} \mathbb{E}_z \left(\|\nabla d_{\Omega}(z)\| - 1 \right)^2$$

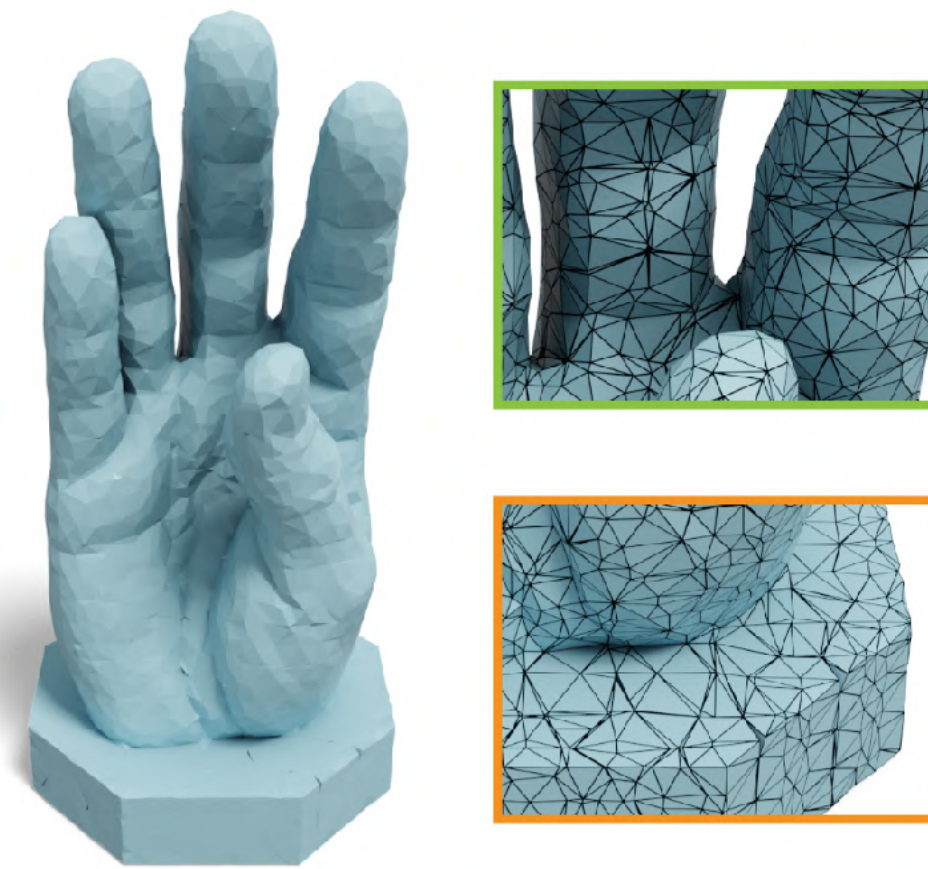


Marching Cubes

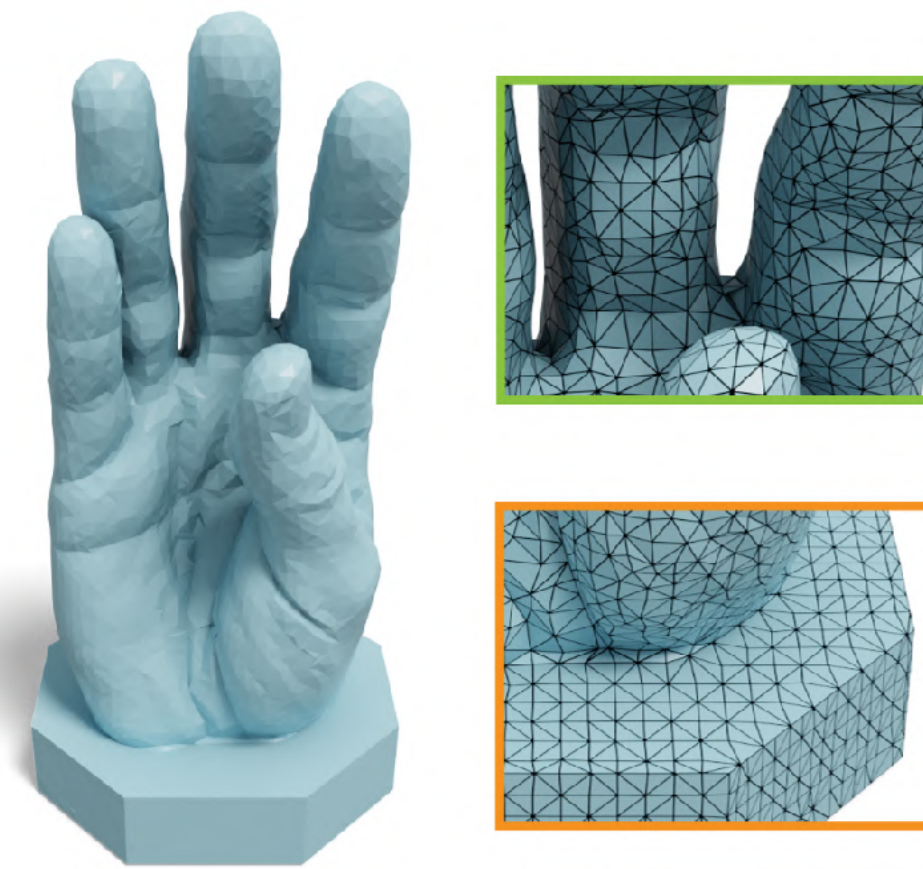
Converting Fields to Polygonal Meshes



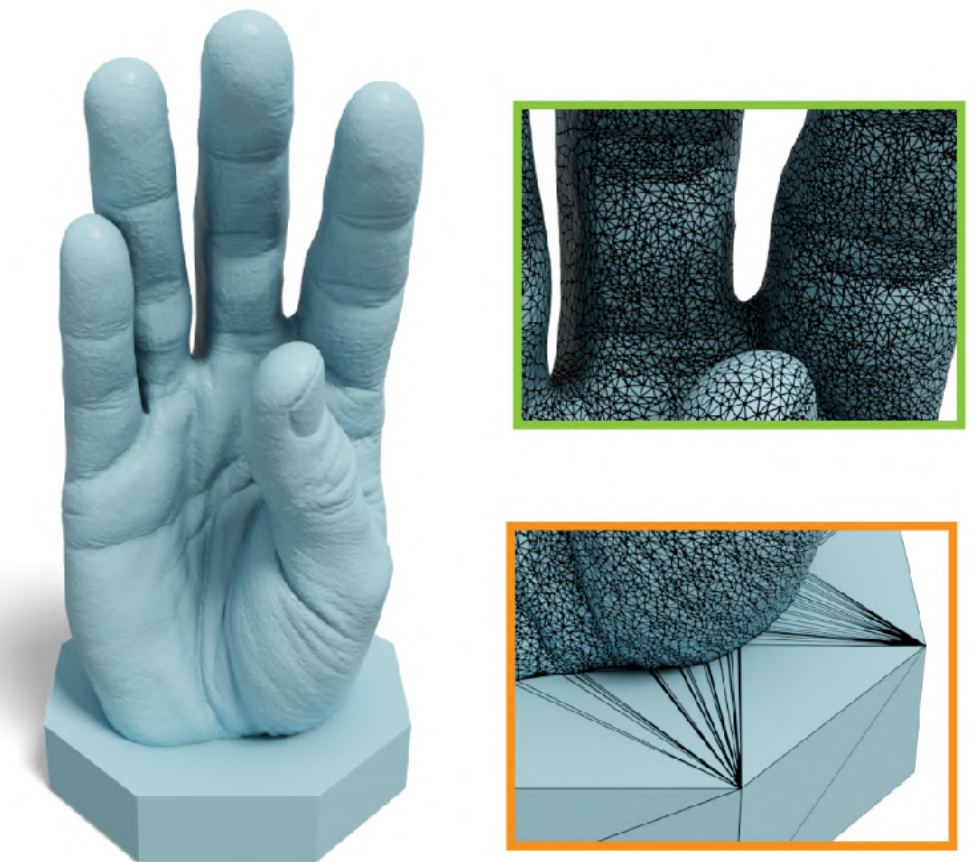
Marching Cubes 15k tris



DMTet 15k tris



FLEXICUBES 13k tris



Reference 91k tris

Key Takeaways

- Fields and volumetric representations for 3D data
 - Voxel, neural and hybrid representations
- Volumetric rendering for novel view synthesis
- Various applications and incremental improvements
 - Amortised models
 - CamP + ZipNeRF still delivers best quality as of May 2024
- Next time: Gaussian Splatting

